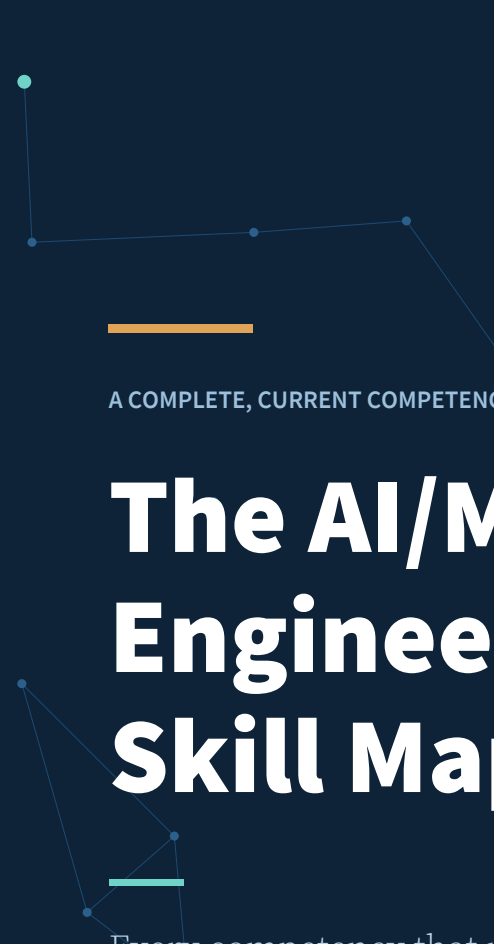
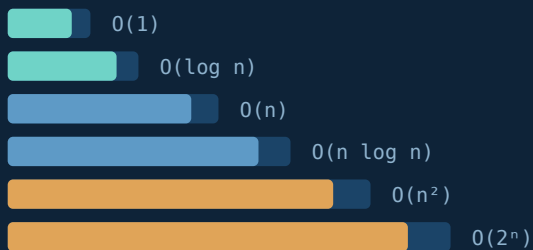




A COMPLETE, CURRENT COMPETENCY MAP

The AI/ML Engineer Skill Map

Every competency that matters in 2026,
what changed, what is obsolete,
and the order to acquire them.



A companion to Data Structures & Algorithms for AI/ML Engineers

First edition · 2026



The AI/ML Engineer Skill Map

Every competency that matters in 2026,
what changed, what is obsolete,
and the order to acquire them.

Most roadmaps list technologies. This one lists capabilities: what you must be able to do, how to tell whether you can do it, what each skill is worth on the job market, and which widely-taught skills have quietly stopped mattering.

A companion to Data Structures & Algorithms for AI/ML Engineers

First edition · 2026

How to Use This Map

This is not a list of technologies. Lists of technologies go stale in a year and, worse, they encourage the wrong behaviour: collecting tool names instead of building capability. This is a list of **capabilities** — things you must be able to *do* — with the tools named only where a specific tool has become the default and knowing it saves you time.

What each chapter contains

Section	What it gives you
WHAT THIS IS FOR	Why the skill exists, in one paragraph, with the failure it prevents
THE CAPABILITIES	A checklist of concrete, testable things you should be able to do
THE TOOLS	What the field has actually standardised on, and what is still contested
HOW TO TELL YOU HAVE IT	A task you can attempt; if you cannot, you do not have the skill
COMMON PITFALL	What people get wrong here, repeatedly
TIME TO COMPETENCE	An honest estimate at roughly 12 hours a week

Each capability is tagged with the level it belongs to:

NOTE | The four tiers used throughout

ESSENTIAL — every AI/ML engineering role expects this. Not having it will end an interview.

CORE — expected in most roles; the difference between junior and mid.

SPECIALIST — deep in one column of the role map; how you become the person others ask.

AWARENESS — you need to know it exists, what it is for, and when to go and learn it. You do not need to be able to do it today.

An honest caveat about currency

Concepts in this map are durable: attention, retrieval, evaluation, quantisation, distributed training, the economics of inference. The *tooling* around them moves faster than any document can. Where a specific library is named it is because it had become the default at the time of writing (2026); before committing a quarter of your learning time to any named tool, check that it is still the one people reach for. The capability outlives the tool, which is why the capability is what this map is organised around.

Where a claim depends on hardware, pricing or a vendor, treat the numbers as orders of magnitude to reason with, not as quotes.

How to actually use it

Do not read it front to back and then start. Read Chapter 1, pick the role you are aiming at, then read Chapter 22 and choose a track. From that point on the map is a reference: you read a chapter when its track week arrives, and you come back to Chapter 24 monthly to re-score yourself.

Build while you read. Every chapter has a **HOW TO TELL YOU HAVE IT** section containing a task. Those tasks are the point. Reading a chapter without attempting its task produces recognition, which feels like knowledge and disappears in a fortnight.

Score yourself honestly. Chapter 24 has a rubric with five levels per area. Most people score themselves one level too high on things they have read about and one level too low on things they have shipped. The correction is to ask “what did I build with this, alone, that broke and that I fixed?”

KEY INSIGHT | The single highest-leverage thing in this document

If you take one thing: **learn to evaluate**. Chapter 15. Almost every team building on foundation models is bottlenecked not by modelling skill but by the inability to tell whether a change made things better. Engineers who can build an honest evaluation harness are, right now, scarce out of proportion to how hard the skill is.

Contents at a Glance

How to Use This Map	iii
Part I — The Landscape	1
1. Five Jobs Behind One Job Title	2
2. The Stack, and Where the Hours Actually Go	6
Part II — Foundations	9
3. Python and Software Engineering	10
4. Mathematics, and Exactly How Much	15
5. Algorithms and Systems	19
Part III — Data	23
6. Data Engineering for ML	24
7. Dataset Curation for Modern Models	29
Part IV — Modelling	33
8. Classical Machine Learning	34
9. Deep Learning	39
10. Training at Scale	44
Part V — Foundation Models	48
11. Foundation Model Fundamentals	49
12. Retrieval and RAG	54
13. Post-Training: Fine-Tuning and Alignment	59
14. Agents and Tool Use	63
15. Evaluation	67

Part VI — Production	73
16. Inference and Serving	74
17. MLOps and Production Practice	79
18. Hardware and Systems	83
Part VII — Judgement	86
19. Safety, Security and Compliance	87
20. Product Sense and Communication	91
Part VIII — Getting There	95
21. What Changed, and What Is Now Obsolete	96
22. Four Learning Tracks	100
23. A Portfolio That Signals Competence	104
24. Self-Assessment	108

Contents

How to Use This Map	iii
----------------------------	------------

Part I — The Landscape	1
-------------------------------	----------

1. Five Jobs Behind One Job Title	2
--	----------

The map	2
---------	---

What transfers, and what does not	4
-----------------------------------	---

2. The Stack, and Where the Hours Actually Go	6
--	----------

The stack	6
-----------	---

Where the hours go	7
--------------------	---

What “knowing” a skill means here	7
-----------------------------------	---

Part II — Foundations	9
------------------------------	----------

3. Python and Software Engineering	10
---	-----------

What this is for	10
------------------	----

The capabilities	10
------------------	----

The tools that have actually standardised	12
---	----

How to tell you have it	13
-------------------------	----

4. Mathematics, and Exactly How Much	15
---	-----------

What this is for	15
------------------	----

The capabilities	15
------------------	----

The honest deferral list	17
--------------------------	----

How to tell you have it	17
-------------------------	----

5. Algorithms and Systems	19
----------------------------------	-----------

What this is for	19
The capabilities	20
Depth required, by role	21
How to tell you have it	22
Part III — Data	23
6. Data Engineering for ML	24
What this is for	24
The capabilities	24
The tools	26
How to tell you have it	27
7. Dataset Curation for Modern Models	29
What this is for	29
The capabilities	29
How to tell you have it	31
Part IV — Modelling	33
8. Classical Machine Learning	34
What this is for	34
The capabilities	34
Tabular or deep?	36
How to tell you have it	37
9. Deep Learning	39
What this is for	39
The capabilities	39
The debugging playbook	41

How to tell you have it	42
10. Training at Scale	44
The arithmetic you do first	44
The capabilities	44
How to tell you have it	46
Part V — Foundation Models	48
11. Foundation Model Fundamentals	49
The capabilities	49
The standard failure modes	51
How to tell you have it	52
12. Retrieval and RAG	54
The pipeline	54
The capabilities	55
When RAG is the wrong answer	57
How to tell you have it	57
13. Post-Training: Fine-Tuning and Alignment	59
The ladder	59
When fine-tuning is actually right	60
The capabilities	60
How to tell you have it	62
14. Agents and Tool Use	63
The loop, and its failure modes	63
The capabilities	63
How to tell you have it	65

15. Evaluation	67
Why this is the bottleneck	67
The loop	67
The capabilities	68
Tools	70
How to tell you have it	71

Part VI — Production **73**

16. Inference and Serving	74
The governing fact	74
The cost ladder	74
The capabilities	75
How to tell you have it	77

17. MLOps and Production Practice	79
The capabilities	79
How to tell you have it	81

18. Hardware and Systems	83
The capabilities	83
How to tell you have it	85

Part VII — Judgement **86**

19. Safety, Security and Compliance	87
Security	87
Safety and fairness	88
Privacy and compliance	89
How to tell you have it	89

20. Product Sense and Communication	91
Problem framing	91
Metrics that connect	92
Writing and speaking	92
How to tell you have it	94
 Part VIII — Getting There	 95
21. What Changed, and What Is Now Obsolete	96
The shape of the change	96
What became important	96
What became less important	97
What did not change	98
Where the demand actually is	98
 22. Four Learning Tracks	 100
The tracks	100
Track A — Software engineer to AI engineer	101
Track B — Data scientist to ML engineer	101
Track C — Student or career changer	102
Track D — ML engineer to research engineer	102
 23. A Portfolio That Signals Competence	 104
What a reviewer is checking	104
Six projects	105
The write-up	106
Where the projects live	107
 24. Self-Assessment	 108

The levels	108
The rubric	109
Reading your scores	110
Turning the rubric into a plan	110

PART I

The Landscape

“AI/ML engineer” now covers five distinct jobs with different skill profiles. Before learning anything, work out which one you are aiming at — the answer changes everything.

-
- 01 Five Jobs Behind One Job Title
 - 02 The Stack, and Where the Hours Actually Go

Five Jobs Behind One Job Title

Decide what you are aiming at before you learn anything

FOUNDATION

PREREQS: none

“AI/ML engineer” is now an umbrella over at least five jobs with materially different skill profiles, different interview loops and different salary bands. Choosing badly costs months. This chapter describes each one honestly — what the day actually looks like, what is tested, and who should aim at it.

WHAT YOU WILL LEARN

- Distinguish the five roles that share this title and what each is measured on
- Identify which skills transfer between them and which do not
- Choose a target role based on your starting point rather than on prestige
- Understand where hiring demand actually is, and where it is contested

The map



Same title on the job board. Very different day jobs — decide which column you want to be strongest in.

Figure 1.1 The same job title, five different centres of gravity. The bars are rough weightings of where the hours go, not of what appears in the job description.

AI Engineer

The day. You build products on top of foundation models you did not train. Retrieval pipelines, agents, structured extraction, evaluation harnesses, prompt and context engineering, guardrails, cost and latency tuning. You touch model weights rarely and model behaviour constantly.

What is tested. Systems design for an LLM application, evaluation design, debugging a bad RAG pipeline, API and product judgement. Increasingly: “here is a failing agent, find out why.”

Who it suits. Software engineers. This is the shortest transition in the field — three to four months of focused work — because the foundation is software engineering, not statistics. It is also where hiring volume has grown fastest.

The risk. It is the easiest role to be shallow in. The market has many people who can wire an API to a vector database; it has few who can tell you whether the result is any good, and fewer who can make it cheap.

ML Engineer

The day. You own models end to end: data, features, training, evaluation, deployment, monitoring. Often the models are gradient-boosted trees over tabular data, because that is what most businesses have. Ranking, fraud, forecasting, churn, pricing, recommendation.

What is tested. Feature engineering and leakage, cross-validation design, metric selection, class imbalance, a production incident postmortem, sometimes a coding round on data manipulation.

Who it suits. Data scientists who want to own production, and software engineers willing to learn statistics properly.

The risk. Under-rated by people chasing the frontier, and consequently less crowded. A great deal of durable business value is here.

ML Platform / Infrastructure Engineer

The day. You build the systems other ML people use: training clusters, feature stores, model registries, serving infrastructure, GPU scheduling, observability. You rarely train a model; you make training and serving fast, cheap and reliable.

What is tested. Distributed systems, Kubernetes and scheduling, performance profiling, storage and networking, reliability engineering.

Who it suits. Backend and infrastructure engineers. The ML-specific content is learnable; the systems depth is not, quickly.

The risk. None much — this is the most consistently underserved role in the field and pays accordingly.

Research Engineer

The day. You implement and scale ideas from papers, run training experiments, write kernels, debug loss curves, build the tooling a research team needs. You may co-author. You are measured on experimental throughput and correctness, not on novelty.

What is tested. Deep learning internals, numerical debugging, reading and reimplementing a paper, distributed training, sometimes CUDA or Triton.

Who it suits. People who genuinely enjoy the details: why the loss spiked at step 40,000, why bf16 changed the result, why the third GPU is idle.

The risk. Concentrated in a small number of labs, so geographically and organisationally narrow. Extremely competitive.

Data Scientist (adjacent, and converging)

The day. Analysis, experimentation, causal questions, dashboards, decision support. Increasingly also: building the evaluation sets that ML and AI engineers depend on, which is a genuine convergence point.

What is tested. Statistics, experiment design, SQL, communication.

Who it suits. People whose comparative advantage is in the question rather than the system.

What transfers, and what does not

From	To	Transfers	You must add
Software engineering	AI engineer	70%	LLM behaviour, retrieval, evals, prompt/context design
Software engineering	ML engineer	45%	statistics, feature engineering, metric design
Data science	ML engineer	55%	engineering rigour, deployment, monitoring
Data science	AI engineer	40%	software engineering, systems, evals at scale
Backend / infra	ML platform	75%	GPU scheduling, training internals, ML-specific storage
ML engineer	Research engineer	50%	deep learning internals, distributed training, kernels
Any	Any	—	the foundations layer: Python, algorithms, systems, maths

KEY INSIGHT | The advice that is actually load-bearing

Pick the role whose *failure modes* you find interesting. You will spend far more time debugging than building, and the debugging is completely different in each column: a bad eval set, a leaking feature, an idle GPU, a diverging loss, an ambiguous business question. If one of those sounds like a puzzle rather than a chore, that is your column.

COMMON PITFALL | Three ways people choose badly

Chasing the frontier. Aiming at research engineer because it sounds prestigious, from a background with no deep learning internals and no systems depth. The transition is real but it is measured in years, not months, and there is a far shorter path to working on interesting problems.

Skipping the foundations layer. Going straight to LLM tooling with weak Python and no algorithms. This works until the first interview with a coding round, or the first production incident, and then it does not.

Optimising for the job title. “AI engineer” pays well in 2026 partly because of scarcity, and scarcity moves. The durable asset is the stack in the next chapter, not the title on the offer letter.

CHAPTER RECAP

- Five jobs share this title: AI engineer, ML engineer, ML platform, research engineer, data scientist. Their day-to-day work, interviews and required depth differ substantially.
- Software engineers convert fastest to AI engineering; backend engineers convert fastest to ML platform; data scientists convert fastest to ML engineering.
- Every path requires the foundations layer, which is the most common point of failure regardless of target.
- Choose by which failure modes you find interesting, because debugging is most of the work.

The Stack, and Where the Hours Actually Go

What the job is made of, versus what courses teach

FOUNDATION

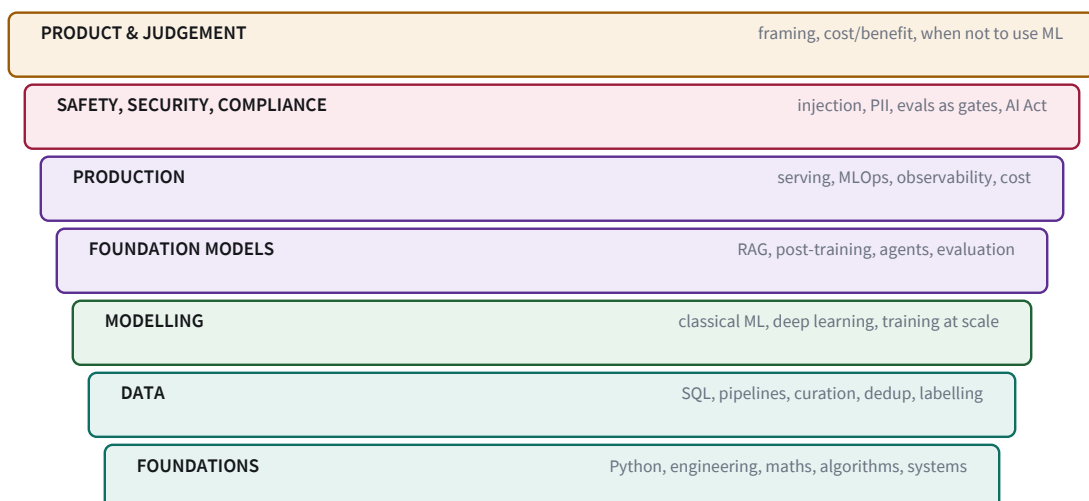
PREREQS: Chapter 1

Before the capability lists, two orienting pictures: the layered stack that every role sits on, and an honest account of where a working engineer's hours go. Both differ sharply from the shape of a typical curriculum, and the gap explains most of the difficulty people have converting study into a job.

WHAT YOU WILL LEARN

- Hold the seven-layer stack in your head and locate any new skill within it
- Understand why the two largest time sinks are the two least taught
- Recognise the difference between a portfolio that signals competence and one that does not

The stack



Read bottom-up. Every hiring loop probes the bottom two layers, and most candidates who fail, fail there.

Figure 2.1 The layers. Nothing above works reliably if a layer below is missing, and interviews probe the bottom two far more than candidates expect.

Read it bottom-up. The property that makes this a *stack* rather than a list is that weakness low down does not stay contained:

- Weak **Python and engineering** shows up as an unmaintainable pipeline, a notebook that cannot be rerun, a subtly non-deterministic result.
- Weak **data** shows up as a model that scores well offline and fails in production, which is usually leakage or train-serve skew.
- Weak **modelling** shows up as reaching for a deep model where a gradient-boosted tree would have been better, faster and explainable.
- Weak **production** shows up as a demo that never ships.
- Weak **judgement** shows up as a technically excellent solution to the wrong problem.

Where the hours go



Proportions are illustrative but the shape is not controversial: the two rows that dominate real work are the two that almost no curriculum teaches.

Figure 2.2 Illustrative proportions, but the shape is not controversial. The two rows that dominate real work — data and evaluation — are the two that almost no curriculum teaches.

This gap is the single best explanation for why people who have completed several courses still struggle in interviews and first jobs. They have spent 65% of their study time on the 15% of the job that is modelling, and have no practice at all in the 50% that is data and evaluation.

KEY INSIGHT | The corrective

For every model you train, make yourself do three things you would rather skip: build the evaluation set *first*, write down the baseline before you start, and deploy it somewhere, however crudely. Those three habits convert study into the shape of the job.

What “knowing” a skill means here

Throughout this map, a capability is written as something you can do without help, on a problem you have not seen. That is a deliberately high bar, and it is the bar interviews use.

Claim	What it actually requires
“I know PyTorch”	You can write a training loop from scratch, debug a NaN, and explain why your GPU is at 40% utilisation

Claim	What it actually requires
“I know RAG”	You can measure retrieval recall separately from answer quality and say which one is broken
“I know fine-tuning”	You can say why you are <i>not</i> fine-tuning for this problem, and be right
“I know Kubernetes”	You can debug a pod that is stuck pending because a GPU node selector is wrong
“I know SQL”	You can read a query plan and explain why a join is slow
“I know evaluation”	You can construct a golden set, a grader, and a significance test for a 3% claimed improvement

COMMON PITFALL | The tutorial ceiling

There is a level you can reach entirely by following tutorials, and it is higher than it used to be — you can build an impressive-looking RAG application without understanding any of it. The ceiling is sharp and it arrives the first time something goes wrong in a way the tutorial did not cover. The exit from the ceiling is always the same: build something with no tutorial for it, and debug it yourself.

CHAPTER RECAP

- The stack has seven layers; weakness low down leaks upward as production incidents rather than staying contained.
- Data work and evaluation dominate real hours and are barely taught; modelling is a minority of the work and most of the curriculum.
- Fix the gap with three habits: eval set first, baseline written down, deploy it somewhere.
- A capability means doing it unaided on an unfamiliar problem, which is the bar interviews use.

PART II

Foundations

The layer nobody can skip. Weakness here caps you at “can follow a tutorial” forever, and it is the most common reason strong candidates fail technical screens.

-
- 03 Python and Software Engineering
 - 04 Mathematics, and Exactly How Much
 - 05 Algorithms and Systems

Python and Software Engineering

The layer that decides whether anything you build survives contact with a team

FOUNDATION

PREREQS: none

Almost every AI/ML engineering interview has a coding round, almost every production incident traces back to engineering practice rather than modelling, and the single clearest signal separating a hobbyist from a hire is whether their code can be run by somebody else next month. This is the least glamorous chapter and the one with the highest expected value.

WHAT YOU WILL LEARN

- Write modern, typed, tested, packaged Python that another engineer can run
- Profile and optimise a Python program rather than guessing
- Use git, CI and containers as a matter of course rather than as a chore
- Know which parts of the modern Python toolchain have actually standardised

What this is for

Machine learning code has a specific failure mode: it works once, on one machine, in one notebook, and then nobody — including its author six weeks later — can reproduce the number. Every practice in this chapter exists to prevent that.

The capabilities

CHECKLIST | ESSENTIAL — modern Python

- Write idiomatic Python: comprehensions, generators, `dataclasses`, context managers, `pathlib`, f-strings, `enumerate` / `zip` / `itertools`
- Use type hints throughout and run a type checker (`mypy` or `pyright`) in CI
- Understand mutability, aliasing, and why a default argument of `[]` is a bug
- Know the cost of the core containers cold (see the DSA companion volume, Chapter 2): `in` on a list is linear, string `+=` in a loop is quadratic
- Use `logging` rather than `print`, with structured fields
- Write generators and iterators for data that does not fit in memory
- Handle errors deliberately: narrow `except`, custom exceptions, no bare `except: pass`

CHECKLIST | ESSENTIAL — testing

- `pytest`: fixtures, parametrisation, `tmp_path`, monkeypatching
- Write a test *before* fixing a bug, so the bug cannot return
- Property-based testing with `hypothesis` for anything numerical or parsing-shaped — it finds the empty input, the duplicate, the NaN
- Test ML code specifically: shape tests, a single-batch overfit test, a determinism test with a fixed seed, an invariance test
- Know what *not* to test: exact float equality, model accuracy in unit tests

CHECKLIST | ESSENTIAL — tooling and environments

- `git` beyond the basics: interactive rebase is not needed, but branching, bisect, reflog and reading a diff are
- Dependency and environment management. `uv` has become the default for new projects (it is dramatically faster and handles Python versions too); `poetry`, `pip-tools` and `conda` all still exist in the wild and you should be able to read a project using any of them
- Lockfiles, and why an unpinned environment is an unreproducible result
- `ruff` for linting and formatting (it has largely absorbed `flake8`, `isort` and `black` for new projects)
- Docker: write a working, layer-cached Dockerfile for a Python service; know why CUDA base images are enormous and how to keep them from being worse
- `make` or `just` so that `make test` works on a fresh clone
- Pre-commit hooks, so that CI failures are rare rather than routine

CHECKLIST | CORE — performance

- Profile before optimising: `cProfile`, `py-spy` (works on a running process), `scalene`, `line_profiler`, `torch.profiler`
- Know the three answers to “Python is slow”: vectorise it (NumPy, Polars), move it out of the loop, or move it out of Python (Numba, Cython, Rust via PyO3, or a C++ extension)
- Concurrency: know when you need threads (I/O bound), processes (CPU bound) or `asyncio` (many concurrent network calls — which describes almost all LLM application code)
- Memory: `tracemalloc`, `memory_profiler`, and the habit of asking how many bytes per element a structure costs

CHECKLIST | CORE — engineering practice

- Write a design document before a non-trivial change: problem, options, choice, tradeoff, how you will know it worked
- Code review: give specific, kind, actionable review; receive it without defensiveness
- Configuration as data (`pydantic`, `hydra`, or plain typed dataclasses) so that experiments are described by a file rather than by edited constants
- Structure a repository so that a stranger can find things: `src/` layout, a runnable `README`, a single entry point per task

CHECKLIST | AWARENESS

- Rust or C++ sufficient to read a kernel or an extension module
- Bash sufficient to read someone's deployment script without fear
- The CPython object model, the GIL, and what free-threaded Python changes

The tools that have actually standardised

Job	Default in 2026	Also common	Notes
Environments and packaging	<code>uv</code>	<code>poetry</code> , <code>conda</code> , <code>pip</code> + <code>venv</code>	<code>uv</code> won new projects on speed; <code>conda</code> persists where CUDA and non-Python deps are entangled
Lint and format	<code>ruff</code>	<code>black</code> + <code>flake8</code> + <code>isort</code>	<code>ruff</code> replaced the trio for most new code
Type checking	<code>mypy</code> , <code>pyright</code>	—	pick one and run it in CI
Testing	<code>pytest</code>	<code>unittest</code>	<code>hypothesis</code> for property tests
Data validation	<code>pydantic</code> v2	<code>attrs</code> , dataclasses	also the schema layer for LLM structured output
Task runner	<code>make</code> , <code>just</code>	shell scripts	any is fine; having one is the point
Containers	Docker	Podman	CUDA images from NVIDIA NGC
CI	GitHub Actions	GitLab CI, Buildkite	run tests, lint, types on every PR

NOTE | Notebooks

Notebooks are excellent for exploration and terrible as the unit of delivery: hidden state, unreviewable diffs, no tests, no imports. The professional pattern is to explore in a notebook and then move anything you will run twice into a module with a test, importing it back into the notebook. Tools like `jupyter` and `nbstripout` make notebooks survivable in git; `papermill` makes them parameterisable. None of that makes a notebook a deliverable.

How to tell you have it

PRACTICE | The task

Take a script you wrote in a notebook. Convert it into a package with a `pyproject.toml`, a locked environment, type hints, three `pytest` tests (one of them a property test), a `Dockerfile`, a `Makefile` with `make test` and `make run`, and a CI workflow that runs lint, types and tests. Then hand the repository to somebody else and watch them try to run it without asking you a question.

If they need to ask you a question, you have not finished.

COMMON PITFALL | The four engineering failures that show up in ML code

1. **Notebook-only delivery.** Nothing is importable, nothing is tested, the result cannot be regenerated.
2. **Unpinned environments.** “It worked last month” is a dependency resolution, not a mystery.
3. **Hidden global state.** A module-level model, a mutable default, a `random` call without a seed. All three produce results that change without the code changing.
4. **Configuration by editing constants.** Experiment provenance becomes impossible, which makes every result unverifiable.

NOTE | Time to competence

From “I can write Python scripts”: **4–6 weeks** at 12 hours a week to reach the ESSENTIAL list solidly. This is the cheapest block of hours in the whole map and it raises the ceiling on everything above it.

CHAPTER RECAP

- The failure mode this chapter prevents is the irreproducible result, and it is the most common failure in applied ML.
- ESSENTIAL: modern typed Python, `pytest` plus `hypothesis`, git, locked environments, `ruff`, Docker, a task runner, CI.
- CORE: profiling before optimising, the right concurrency model (usually `asyncio` for LLM application code), design documents, reviewable structure.
- Notebooks are for exploration; anything run twice moves into a tested module.
- The test of the skill is whether a stranger can clone and run your work without asking you anything.

Mathematics, and Exactly How Much

The parts you will use, and the parts you can safely defer

FOUNDATION

PREREQS: none

The maths requirement for AI/ML engineering is widely overstated and widely understated at the same time. You do not need measure theory. You absolutely do need to be fluent in linear algebra as shapes, in probability as uncertainty you must quantify, and in optimisation as why your loss is not going down. This chapter separates the two lists honestly.

WHAT YOU WILL LEARN

- Reason about tensor shapes, ranks and matrix operations without hesitation
- Use probability and statistics to make honest claims about model performance
- Understand gradients and optimisation well enough to debug training
- Know which mathematics you can defer indefinitely, and why

What this is for

Three concrete failures that mathematics prevents, in ascending order of frequency:

1. You cannot debug a shape error, so you guess at `transpose` and `reshape` until it runs — and then the model trains on the wrong axis.
2. You report a 2% improvement that is noise, and the team ships it.
3. Your loss plateaus and you do not know whether the cause is the learning rate, the initialisation, the data, or a bug.

The capabilities

CHECKLIST | ESSENTIAL — linear algebra

- Vectors, matrices, tensors as *shapes*: you can predict the output shape of any composition of operations before running it
- Matrix multiplication, transpose, broadcasting rules, batched matmul
- Dot products and cosine similarity, and why normalisation changes the ranking
- Norms: L1, L2, and what each does as a regulariser
- Eigenvalues and eigenvectors *conceptually*: PCA, spectral clustering, and what “the top singular vectors” means
- SVD and low-rank approximation — this is literally what LoRA is
- Why numerical precision matters: fp32 vs bf16 vs fp16 range and precision, and where catastrophic cancellation appears

CHECKLIST | ESSENTIAL — probability and statistics

- Distributions you will meet: Bernoulli, categorical, Gaussian, Poisson, log-normal, power law (and knowing that real data is usually the last two)
- Expectation, variance, covariance; the law of large numbers and the CLT as intuitions rather than proofs
- Conditional probability and Bayes’ rule, fluently
- Maximum likelihood, and why cross-entropy is the negative log-likelihood
- KL divergence and entropy: they appear in distillation, in variational methods, in DPO, and in drift detection
- Confidence intervals and bootstrapping — how you say “2% better” honestly
- Hypothesis testing and multiple-comparison correction: if you evaluate 20 prompts you will find a winner by chance
- Sampling: with and without replacement, stratification, and how biased sampling silently invalidates an evaluation

CHECKLIST | ESSENTIAL — calculus and optimisation

- Derivatives, partial derivatives, the chain rule — because backpropagation is the chain rule applied to a graph
- Gradients and what a gradient step does geometrically
- Convexity, and why it matters that deep learning is not convex
- Gradient descent and its variants: momentum, Adam/AdamW, and what weight decay actually does
- Learning-rate schedules and warmup, and why a large batch needs a different learning rate
- Why gradients vanish and explode, and the standard mitigations: normalisation, residual connections, clipping, careful initialisation

CHECKLIST | CORE

- Information theory: entropy, cross-entropy, mutual information, perplexity — and being able to explain why perplexity is $e^{\text{cross-entropy}}$
- Numerical methods: conditioning, stability, the log-sum-exp trick, why you work in log space
- Matrix calculus enough to read a paper's gradient derivation
- Experimental design: power analysis, A/B test sizing, sequential testing hazards, CUPED-style variance reduction

CHECKLIST | AWARENESS

- Convex optimisation theory, Lagrangians, duality
- Measure-theoretic probability
- Stochastic differential equations (relevant if you work on diffusion)
- Category theory, topology, and everything else occasionally claimed to be required. It is not.

The honest deferral list

Often demanded	Actually needed for	Defer if
Measure theory	writing probability theory papers	you are building systems
Convex optimisation proofs	optimisation research	you use AdamW like everyone else
Manual backprop derivations	understanding, once	you have done it once by hand for a 2-layer net
Full SVD algorithms	numerical linear algebra research	you call <code>np.linalg.svd</code>
Real analysis	proofs	always, for engineering roles

KEY INSIGHT | The reframing that makes this tractable

Learn the mathematics *as debugging tools*, not as a course. Linear algebra is how you reason about shapes. Probability is how you avoid claiming a result you do not have. Optimisation is how you explain a flat loss curve. Studied that way each topic has an immediate payoff, which is the only reliable way adults retain mathematics.

How to tell you have it

PRACTICE | The tasks

1. Given x of shape (B, L, D) , weights $(D, 4D)$ and $(4D, D)$, and a reshape into $(B, L, H, D/H)$, write down every intermediate shape in a transformer block from memory, including the attention matrix.
2. Implement backpropagation by hand for a two-layer MLP — no autograd — and match PyTorch's gradients to within float tolerance.
3. Your model scores 0.842 and the baseline scores 0.826 on a 2,000-example test set. Compute a bootstrap confidence interval and state whether you would ship it. Then compute how large the test set must be for a 1% difference to be detectable.
4. Explain why `softmax` is implemented by subtracting the row maximum, and construct an input where the naive version returns `nan`.

COMMON PITFALL | The three that actually bite

Claiming an improvement that is noise. The most common statistical failure in applied ML, and the one that costs the most: teams ship changes that do nothing and lose the ability to tell which past change helped.

Evaluating many things and reporting the best. Twenty prompts on one test set will produce a 2-sigma winner by chance. Hold out a second set, or correct for the comparisons.

Trusting a metric without knowing its variance. Report a confidence interval alongside every headline number, every time. It changes decisions.

NOTE | Time to competence

From a typical engineering degree: **4–6 weeks** to firm up the ESSENTIAL lists. From no mathematical background: **3–4 months**, and it is worth every hour. Learn it alongside the code, not before it.

CHAPTER RECAP

- Linear algebra as shapes, probability as honest uncertainty, optimisation as training debugging. That framing is what makes it stick.
- ESSENTIAL: shapes and broadcasting, norms, SVD conceptually, distributions, Bayes, MLE and cross-entropy, KL, bootstrapping, significance, gradients, AdamW, schedules, vanishing/exploding gradients.
- Derive backprop by hand once; call `np.linalg.svd` forever after.
- Measure theory, real analysis and convex optimisation proofs are not required for engineering roles.
- The highest-value statistical habit is reporting a confidence interval with every number.

Algorithms and Systems

Why the interview tests it, and why the job needs it more

FOUNDATION

PREREQS: Chapter 3

Data structures and algorithms are tested in interviews for reasons that are only partly about interviews. The same material determines whether your deduplication job finishes, whether your retrieval index fits in memory, and whether your dataloader starves the GPU. This chapter states the requirement precisely and points at the companion volume that covers it.

WHAT YOU WILL LEARN

- Know exactly which algorithmic material is expected, and to what depth
- Reason about the memory hierarchy well enough to explain a 10x performance gap
- Understand operating-system and networking fundamentals at the level production requires
- Apply the specific structures that recur in ML systems

What this is for

Two distinct reasons, and it is worth separating them.

The interview reason. Most ML and AI engineering loops include a coding round at roughly the level of arrays, hashing, two pointers, trees, heaps, graphs and dynamic programming. This is not going away, and it is the most mechanically preparable part of any loop.

The job reason, which is larger. The algorithmic content of a modern ML system is substantial and mostly invisible until it breaks: deduplicating a corpus is locality-sensitive hashing; a tokenizer is a trie and a priority queue; prioritized replay is a Fenwick tree; vector search is quantisation and a navigable graph; a KV cache manager is a paging system; distributed gradient reduction is a ring algorithm with a provable bandwidth bound.

The capabilities

CHECKLIST | ESSENTIAL — algorithms

- Complexity analysis, including amortised, and the discipline of stating time *and* space
- Arrays, hashing, sets, and the reflex that `x in list` is linear
- Sorting, binary search — including binary search over an answer space, which is the form that actually appears at work
- Two pointers and sliding windows
- Trees: traversal, binary search trees, balance
- Heaps and top-*k*, including top-*k* over a stream with bounded memory
- Graphs: representations, BFS, DFS, topological sort, shortest paths
- Dynamic programming: recognise it, formulate the state, write it both top-down and bottom-up
- Recursion, and converting it to iteration when depth is a risk

CHECKLIST | CORE — systems

- The memory hierarchy: cache lines, why sequential access is an order of magnitude faster than random, why a linked list underperforms its asymptotics
- Arithmetic intensity: FLOPs per byte moved, and the roofline test for whether an optimisation can possibly help
- Processes, threads, the GIL, and which concurrency model fits which problem
- Virtual memory, page cache, memory mapping — which is how large datasets are actually read
- Networking enough to reason about latency budgets: round trips, TLS handshakes, HTTP/2 multiplexing, gRPC, why a chatty client is slow
- File formats and their access patterns: Parquet, Arrow, and why columnar beats row-oriented for analytics

CHECKLIST | CORE — the ML-specific structures

Structure	Where it appears
Hash set / Bloom filter	exact and approximate deduplication, contamination checks
MinHash + LSH	near-duplicate detection in a training corpus
Trie, Aho–Corasick	tokenizers, PII scrubbing, constrained decoding
Heap	top- <i>k</i> retrieval, beam search, BPE training, schedulers
Fenwick / segment tree	prioritized replay, dynamic weighted sampling
Ring buffer	replay buffers, sliding-window KV caches, streaming windows
Union–find	turning pairwise near-duplicates into clusters
Radix / prefix tree	KV-cache prefix sharing in LLM serving
Product quantisation, HNSW	vector indexes
Count–Min, HyperLogLog	stream statistics at corpus scale

CHECKLIST | AWARENESS

- Database internals: B+ trees, LSM trees, query planning
- Distributed systems theory: consistency models, consensus, partitioning
- Compiler basics, enough to understand what `torch.compile` does

Depth required, by role

Role	Interview depth	Job depth
AI engineer	medium — expect a coding round	medium: retrieval, caching, cost
ML engineer	medium	medium: data processing at scale
ML platform	high	high: this <i>is</i> the job
Research engineer	medium–high	high: kernels, memory, parallelism
Data scientist	low–medium	low

KEY INSIGHT | The companion volume

This map’s sibling — *Data Structures & Algorithms for AI/ML Engineers* — covers exactly this material in 47 chapters, from complexity and the memory hierarchy through to paged attention, ring all-reduce and FlashAttention as an IO-aware algorithm. Every structure in the table above has a chapter. If you are working through this map, that volume is the Part II reading.

How to tell you have it

PRACTICE | The tasks

1. Solve 50–75 problems across the standard patterns, timed, without looking at solutions first. Track which patterns you fail.
2. Explain, without notes, why summing a 4096×4096 matrix along axis 0 and along axis 1 differ by 5–10x in wall-clock time.
3. Given 100 million documents and 32 GB of RAM, design a deduplication pipeline. State the structures, the memory arithmetic, and what your approximation costs you in dropped unique documents.
4. Profile a training loop that is at 40% GPU utilisation and find the bottleneck. (It will be the input pipeline. Prove it.)

COMMON PITFALL | Grinding problems without patterns

Solving 300 problems by memorising 300 solutions produces nothing durable. Solving 60 while explicitly naming the pattern each one belongs to, and reimplementing the pattern from memory a day later, produces recall that survives. The catalogue of patterns matters more than the count of problems.

NOTE | Time to competence

Interview-ready from a software background: **5–6 weeks** at 12 hours. Job-ready systems intuition: continuous, but the memory-hierarchy chapter alone is a weekend and changes how you read every profile afterwards.

CHAPTER RECAP

- Algorithms are tested because loops test them, and needed because ML systems are built out of them.
- **ESSENTIAL:** complexity, hashing, sorting and binary search (including over an answer space), windows, trees, heaps, graphs, dynamic programming.
- **CORE** systems: the memory hierarchy, arithmetic intensity, concurrency models, memory mapping, latency budgets, columnar formats.
- Ten specific structures recur throughout ML systems; learn them by the system that uses them.
- Practise by pattern, not by problem count.

PART III

Data

Models are commodities; data pipelines are not. This is where most of the hours go and where most of the irreproducible results come from.

-
- 06 Data Engineering for ML
 - 07 Dataset Curation for Modern Models

Data Engineering for ML

Thirty per cent of the hours, and most of the bugs that reach production

CORE

PREREQS: Chapter 3

Every model is a function of its data, and most model failures are data failures wearing a modelling costume. This chapter covers the data skills an ML or AI engineer is actually expected to have — not the full data-engineering role, but the substantial overlap that you will be held responsible for.

WHAT YOU WILL LEARN

- Write SQL that a data engineer would not rewrite
- Choose and use the modern dataframe and columnar tooling correctly
- Build a pipeline that is idempotent, observable and backfillable
- Prevent leakage and train-serve skew structurally rather than by vigilance

What this is for

The three most expensive bugs in applied ML are all data bugs:

- **Leakage:** information from the future, or from the label, reaches the features. The offline metric is excellent and the production metric is not.
- **Train-serve skew:** the feature computed at training time is not the feature computed at serving time. Nothing errors; the model is just wrong.
- **Silent distribution change:** an upstream system starts sending nulls, or a unit changes from cents to dollars, and no test catches it.

None of these is detectable by looking at the model.

The capabilities

CHECKLIST | ESSENTIAL — SQL

- Joins of every kind, and predicting which produces duplicate rows
- Window functions: `ROW_NUMBER`, `LAG`, `LEAD`, running aggregates, partitioned ranking. This is the single most useful SQL skill for ML work
- CTEs, and structuring a long query so it can be read
- Aggregation with `GROUP BY`, `HAVING`, `FILTER`
- Reading a query plan and explaining why something is slow
- Point-in-time correct joins — the as-of join that prevents leakage

CHECKLIST | ESSENTIAL — dataframes and formats

- `pandas` fluently, including the traps: `SettingWithCopyWarning`, silent dtype promotion to `object`, index alignment surprises
- `polars` or `duckdb` for anything that strains `pandas`. Both are substantially faster, use less memory and have saner semantics; DuckDB in particular lets you run SQL directly over Parquet files with no server
- Parquet and Arrow: columnar layout, predicate and projection pushdown, compression, row groups, and why columnar is right for training data
- Memory-mapped arrays for datasets larger than RAM
- Partitioning schemes, and why `date=2026-01-01/` directories make queries cheap

CHECKLIST | CORE — pipelines

- An orchestrator: Airflow, Dagster or Prefect. Dagster's asset model maps particularly well onto ML dependencies; Airflow is the most widely deployed
- Idempotency: rerunning a task must produce the same result, not duplicate rows
- Backfills, and designing partitions so that a backfill is possible at all
- Data contracts and schema enforcement at boundaries
- Incremental processing rather than full recomputation
- `dbt` if your organisation has a warehouse

CHECKLIST | CORE — data quality

- Validation as code: Great Expectations, `pandera`, or plain assertions that run in the pipeline and fail loudly
- The checks that matter: row counts within expected bounds, null rates, cardinality, range, referential integrity, freshness
- Distribution monitoring on *inputs*, not only on outputs
- Knowing the difference between a broken pipeline and a changed world, and alerting differently for each

CHECKLIST | CORE — feature engineering and serving

- Feature stores conceptually: offline store for training scans, online store for point lookups, and one definition shared between them
- Point-in-time correctness — computing a feature as it was at the label’s timestamp, not as it is now
- Why train–serve skew arises: two implementations of one feature, one in batch SQL and one in a streaming service
- Windowed aggregates, and stating the window semantics precisely

CHECKLIST | AWARENESS

- Streaming: Kafka, Flink, and when a real-time feature is genuinely needed (usually less often than proposed)
- Lakehouse table formats: Iceberg, Delta Lake, Hudi — time travel, schema evolution, ACID on object storage
- CDC (change data capture) from operational databases

The tools

Job	Reach for	When
Ad-hoc analysis, < 1 GB	pandas	familiarity wins
1–100 GB on one machine	Polars or DuckDB	5–50x faster, far less memory
SQL over files, no server	DuckDB	genuinely excellent; use it more
> 1 TB, cluster	Spark, Ray Data, BigQuery	distributed is a cost, not a default
Orchestration	Dagster, Airflow, Prefect	Dagster for asset-shaped ML work
Validation	pandera, Great Expectations	run it in the pipeline, not in a notebook
Storage	Parquet on object storage	plus Iceberg/Delta if you need ACID
Feature store	Feast, or your platform’s	or a well-disciplined table; the discipline is the hard part

KEY INSIGHT | The pandas-to-Polars transition is worth making

Not for fashion: Polars has a lazy API that optimises the whole query, a columnar engine that parallelises across cores by default, and semantics without pandas' index surprises. DuckDB does the same job through SQL and reads Parquet directly. For a dataset that makes pandas swap, either one typically turns a job that did not finish into one that takes ninety seconds. Learn one; keep pandas for small interactive work and for the enormous ecosystem that expects it.

How to tell you have it

PRACTICE | The task

Build a small feature pipeline end to end:

1. Raw events in Parquet, partitioned by date.
2. A point-in-time-correct join producing `(entity, label_time, label, features)` where every feature uses only data strictly before `label_time`.
3. Validation that fails the run if null rates or ranges move.
4. An online path that recomputes the same features from the same definition for a single entity at serving time.
5. A test that asserts the training-time and serving-time features agree for a sample of entities.

Step 5 is the one that matters and the one nearly everybody skips.

COMMON PITFALL | Leakage, in the four shapes it actually takes

1. **Target leakage.** A feature computed after the label event, such as “number of support tickets” for a churn label.
2. **Temporal leakage.** Random train/test split on time-series data. Always split by time.
3. **Group leakage.** The same user, patient or document appears in both splits. Split by group.
4. **Preprocessing leakage.** Scaling, imputation or feature selection fitted on the full dataset before splitting. Fit inside the fold.

Each produces an offline metric that is too good and a production metric that is disappointing — and the gap is never attributed to the right cause.

NOTE | Time to competence

6–8 weeks for the ESSENTIAL and CORE lists, assuming you are writing SQL weekly. The point-in-time join is the single highest-value thing in the chapter and deserves a full week on its own.

CHAPTER RECAP

- Data bugs — leakage, train-serve skew, silent distribution change — are the most expensive failures in applied ML and are invisible from the model side.
- ESSENTIAL: SQL with window functions and as-of joins; pandas plus Polars or DuckDB; Parquet and Arrow.
- CORE: an orchestrator, idempotency, backfills, validation as code that fails the run, feature definitions shared between training and serving.
- Reach for a cluster only when one machine with DuckDB or Polars has genuinely failed.
- Leakage has four shapes; all four are prevented structurally by splitting and fitting correctly, not by care.

Dataset Curation for Modern Models

The skill that improved models more than architecture did

CORE

PREREQS: Chapter 6

Over the last several years the largest reliable gains in model quality have come from better data rather than better architectures. Deduplication, filtering, mixture design, synthetic generation and contamination control are now engineering disciplines with known techniques. This chapter covers them at the level a practitioner needs.

WHAT YOU WILL LEARN

- Deduplicate a corpus exactly and approximately, at scale
- Design and measure a data mixture rather than guessing one
- Generate and validate synthetic data without collapsing the distribution
- Detect and report benchmark contamination
- Run a labelling operation that produces measurable agreement

What this is for

Three results have become consensus and should shape how you spend time: deduplicated data trains better models per token; filtered data beats more data past a point; and mixture proportions matter as much as total volume. All three are data-engineering work, not modelling work.

The capabilities

CHECKLIST | CORE — deduplication

- Exact dedup with content hashing, and why you store a 16-byte digest rather than the document
- Near-duplicate detection with MinHash and LSH: shingling, signature length, banding, and choosing (b, r) for a target similarity threshold
- SimHash and Hamming-distance blocking for cosine-style similarity
- Clustering candidate pairs with union-find, and capping cluster size so transitive chaining does not merge an entire topic
- Granularity: document-level versus paragraph- or line-level, and the tradeoff (line-level removes far more boilerplate and risks fragmenting documents)
- Which copy to keep — often a bigger quality lever than the threshold

CHECKLIST | CORE — filtering and quality

- Heuristic filters: length, symbol ratio, repetition, language identification, perplexity under a reference model
- Classifier-based filtering: train a small model to score “is this like my high-quality reference set?” and threshold on it
- The boilerplate problem: navigation text, licence blocks, cookie notices
- Toxicity, PII and licence filtering, and the fact that each filter has a measurable cost in recall of useful content
- Measuring the effect of a filter *by training something*, not by inspection

CHECKLIST | CORE — mixture and curriculum

- Domain mixture as an explicit, versioned artefact: proportions per source
- Upsampling and downsampling policies, and epoch counts per source
- Measuring mixture effects with small proxy models before committing compute
- Curriculum: ordering by difficulty or quality, and honest scepticism about how much it helps

CHECKLIST | CORE — synthetic data

- Generation patterns: instruction synthesis, self-instruct style expansion, distillation from a stronger model, back-translation, programmatic generation with verifiable answers
- Verification: the highest-value synthetic data is data where correctness can be *checked* (code that runs, maths with a known answer, format that parses)
- Diversity control: temperature, seed prompts, persona conditioning, and measuring coverage rather than assuming it
- Model collapse: training repeatedly on your own outputs narrows the distribution. Mitigate by anchoring to real data, by filtering aggressively, and by measuring diversity explicitly
- Licensing and terms-of-service constraints on distilling from a commercial model — this is a real legal question, not a formality

CHECKLIST | CORE — contamination

- Detect test-set leakage into training data with n -gram hashing plus an exact check, or a suffix array for arbitrary-length matching
- Report coverage per test example, not just a binary flag
- Decide and state whether you matched tokens or characters; the numbers are not comparable otherwise
- Re-evaluate on a decontaminated subset and report both numbers

CHECKLIST | CORE — labelling

- Writing annotation guidelines that two people interpret the same way — this is harder and more valuable than it sounds
- Inter-annotator agreement: Cohen's/Fleiss' kappa, Krippendorff's alpha, and what an acceptable value is for your task
- Adjudication of disagreements, and treating disagreement as signal about the guidelines rather than about the annotators
- Active learning: label where the model is uncertain or where disagreement is high
- Weak supervision and programmatic labelling as a first pass
- LLM-assisted labelling with human verification — now the default for many tasks, with the caveat that the model's biases become the dataset's biases

How to tell you have it

PRACTICE | The task

Take a corpus of at least a million documents. Then:

1. Report the exact-duplicate rate and the near-duplicate rate at Jaccard 0.8, with the (b, r) you chose and why.
2. Apply three filters and report, for each, how many documents it removed and a sample of twenty it removed — half of which you should read.
3. Build a held-out evaluation set and check it for contamination against the training corpus. Report coverage.
4. Train the same small model on the raw and the curated corpus and report the difference with a confidence interval.

Step 4 is the whole point. Curation claims without a trained comparison are opinions.

COMMON PITFALL | Four curation mistakes with expensive consequences

Deduplicating after splitting. Near-duplicates spread across train and test inflate every metric. Deduplicate first, split second.

Filtering without reading. Every filter removes things you did not intend to remove. Read a random sample of what each filter kills, every time.

Transitive over-merging. Near-duplicate is not a transitive relation; a chain of 0.85-similar documents can merge an entire topic into one cluster. Cap cluster size.

Synthetic data without verification. Generating a million examples from a model and training on them without checking correctness amplifies the generator's errors and narrows the distribution simultaneously.

NOTE | Time to competence

4–6 weeks to be able to run the pipeline above at moderate scale. Deduplication mechanics are a week; the judgement about filters and mixtures takes longer and is learned by training models and comparing.

CHAPTER RECAP

- Data curation has produced larger reliable gains than architecture work; treat it as a primary skill.
- Dedup exactly with hashes and approximately with MinHash/LSH plus union-find; cap cluster size and decide granularity deliberately.
- Filter with heuristics and classifiers, and always read a sample of what each filter removed.
- Mixture proportions are a versioned artefact; measure them with proxy models.
- Synthetic data is most valuable where correctness is checkable; guard against collapse by anchoring to real data and measuring diversity.
- Check contamination, report coverage, and state token versus character matching.
- Labelling quality is a guidelines problem measured by inter-annotator agreement.

PART IV

Modelling

Classical machine learning still wins on tabular data and still pays well. Deep learning is the other half. You need both, and you need to know which one a problem wants.

-
- 08 Classical Machine Learning
 - 09 Deep Learning
 - 10 Training at Scale

Classical Machine Learning

Still the right answer for most business problems, and still interviewed

CORE

PREREQS: Chapters 4, 6

Gradient-boosted trees remain the strongest method for tabular data, which is what most organisations actually have. Classical ML is also where the discipline of honest evaluation was developed, and those habits transfer directly to foundation-model work. Skipping this layer is the most common gap in engineers who learned ML after 2022.

WHAT YOU WILL LEARN

- Choose and tune the right classical model for a tabular problem
- Design a validation scheme that does not lie to you
- Select and interpret metrics appropriate to the decision being made
- Calibrate probabilities and explain model behaviour to a non-specialist

What this is for

If the data is a table of tens of thousands to tens of millions of rows with heterogeneous columns — which describes fraud, churn, pricing, credit, forecasting, ranking features and most of what a business measures — a gradient-boosted tree will usually beat a neural network, train in minutes rather than hours, need no normalisation, handle missing values natively, and be explainable. That is not nostalgia; it is repeatedly reproduced.

The capabilities

CHECKLIST | ESSENTIAL — models

- Linear and logistic regression, with regularisation, as the baseline you must beat and the model you can always explain
- Gradient-boosted trees: XGBoost, LightGBM, CatBoost. Know the differences — LightGBM's leaf-wise growth and histogram binning, CatBoost's ordered target statistics for categoricals
- Random forests, and why boosting usually beats bagging on tabular data
- k -NN, k -means, DBSCAN/HDBSCAN, and hierarchical clustering
- PCA and UMAP for dimensionality reduction and visualisation, with the caveat that UMAP distances between clusters are not meaningful
- Isolation Forest and similar for anomaly detection

CHECKLIST | ESSENTIAL — validation

- Train/validation/test discipline, and never touching test until the end
- k -fold, stratified k -fold, group k -fold, and time-series splits — choosing the right one is most of the skill
- Nested cross-validation when you are selecting hyperparameters *and* estimating performance
- Preventing the four leakages from Chapter 6, structurally
- The habit of a fixed, versioned holdout that predates any modelling

CHECKLIST | ESSENTIAL — metrics

- Classification: accuracy (rarely the right choice), precision, recall, F1, ROC AUC, PR AUC. Know that PR AUC is the right one under heavy imbalance
- Choosing an operating point from a cost matrix rather than from a default 0.5 threshold
- Calibration: reliability diagrams, Brier score, Platt scaling, isotonic regression — essential whenever the probability itself is used in a downstream decision
- Regression: MAE, RMSE, MAPE and its failure at small values, quantile loss for intervals
- Ranking: NDCG, MRR, MAP, recall@ k — and knowing which matches the product
- Reporting a confidence interval with every number

CHECKLIST | CORE — practice

- Feature engineering for tabular data: target encoding done inside the fold, interactions, binning, cyclical encodings for time
- Class imbalance: class weights, threshold tuning, and honest scepticism about SMOTE (it frequently does nothing once the threshold is tuned)
- Hyperparameter search: random search beats grid; Optuna's Bayesian search beats random; early stopping on a validation fold is non-negotiable
- Interpretability: SHAP values, permutation importance, partial dependence. Know that tree feature importances are biased toward high-cardinality features
- Model selection under a latency or memory budget, not just a metric

CHECKLIST | AWARENESS

- Causal inference: uplift modelling, propensity scores, difference-in- differences. The question “does this feature cause the outcome” is different from “does it predict it”, and businesses usually want the first
- Survival analysis for time-to-event problems
- Conformal prediction for distribution-free prediction intervals — a genuinely useful, underused technique

Tabular or deep?

Situation	Choose	Why
Tabular, < 10M rows	GBDT	faster, better, explainable, no preprocessing
Tabular, very high cardinality categoricals	GBDT (CatBoost) or embeddings + MLP	depends on interaction depth needed
Tabular + text or image columns	embed the unstructured column, then GBDT	hybrid beats both pure approaches
Images, audio, text	deep	the inductive biases matter
Sequences with long dependencies	deep	
Very small data (< 1000 rows)	linear or GBDT with heavy regularisation	deep overfits
Need a probability you will act on	anything, then calibrate	calibration is a separate step

KEY INSIGHT | Why this chapter still matters if you want to do LLM work

The evaluation discipline transfers wholesale. Golden sets, held-out data, confidence intervals, threshold selection from a cost matrix, calibration, the refusal to trust a single number — all of it was developed here and all of it is exactly what foundation-model teams are currently missing. Engineers who learned evaluation on tabular problems are noticeably better at evaluating LLM systems.

How to tell you have it

PRACTICE | The task

Take an imbalanced tabular dataset with a temporal component. Then:

1. Build a baseline (majority class, then logistic regression) and write down its number before doing anything else.
2. Design the validation scheme and justify it in two sentences. If the data has time, you must not use random k -fold.
3. Train a GBDT, tune with Optuna and early stopping.
4. Select a threshold from an explicit cost matrix, not from 0.5.
5. Calibrate and produce a reliability diagram.
6. Report the improvement over baseline with a bootstrap confidence interval.
7. Explain the top five features to somebody non-technical using SHAP, and have them tell you whether the explanation is plausible.

Step 7 catches leakage more often than any test does.

COMMON PITFALL | The five that recur

1. **Accuracy on imbalanced data.** 99% accuracy on a 1% positive rate is the majority-class baseline.
2. **Random split on time-series data.** The model learns from the future.
3. **Target encoding fitted outside the fold.** Leakage, and a dramatic offline improvement that vanishes in production.
4. **Trusting tree feature importances.** They are biased; use permutation importance or SHAP.
5. **Uncalibrated probabilities used in a decision rule.** A model that ranks perfectly can still produce probabilities that are systematically wrong, and expected-value calculations built on them will be wrong too.

NOTE | Time to competence

6–8 weeks for ESSENTIAL and CORE with real datasets. The validation and metric material is the part that transfers everywhere and deserves the larger share of the time.

CHAPTER RECAP

- GBDT is still the default for tabular data: faster, stronger, explainable, and tolerant of missing values and raw categoricals.
- The validation scheme is most of the skill; choose it from the structure of the data (time, groups, imbalance).
- Choose metrics from the decision, choose the threshold from a cost matrix, and calibrate whenever the probability is acted on.
- Embedding an unstructured column and feeding it to a GBDT is an underused, strong hybrid.
- The evaluation discipline learned here is precisely what foundation-model teams most often lack.

Deep Learning

Enough internals to debug it, not just to call it

CORE

PREREQS: Chapters 4, 8

You can use deep learning productively without being able to derive it, but you cannot debug it that way — and debugging is the job. This chapter covers the architecture knowledge, the training mechanics and the failure modes that let you look at a loss curve and form a hypothesis rather than a superstition.

WHAT YOU WILL LEARN

- Implement a transformer block from scratch and predict every tensor shape
- Write a training loop and debug it when the loss does not move
- Know the modern architectural components and what problem each one solves
- Use mixed precision, checkpointing and compilation correctly

What this is for

The difference between an engineer who can use PyTorch and one who can *debug* PyTorch is worth a level. The second one looks at a flat loss curve and says “check the learning rate, then check whether the labels are shuffled relative to the inputs, then overfit a single batch” — and is right within three tries.

The capabilities

CHECKLIST | ESSENTIAL — frameworks

- PyTorch fluently: `nn.Module`, `autograd`, `DataLoader`, `devices`, `state_dict`
- Writing a training loop by hand before reaching for a framework wrapper
- `torch.compile`, what it does (graph capture, fusion, kernel selection), and why graph breaks cost you the benefit
- Mixed precision: `bf16` as the default for training where supported, `fp16` with a gradient scaler where not, and why `bf16` is preferred (same exponent range as `fp32`, so no scaler)
- Debugging: `detect_anomaly`, hooks, `torch.profiler`, and reading a trace
- Hugging Face `transformers`, `datasets`, `accelerate`, `peft`, `trl` — the practical ecosystem
- JAX at AWARENESS level: `jit`, `vmap`, `pmap`, functional style. Dominant in some labs and on TPUs, a minority elsewhere

CHECKLIST | ESSENTIAL — architectures

- The transformer, completely: embeddings, positional information, scaled dot-product attention, multi-head attention, the feed-forward block, residual connections, normalisation, the output head
- Modern components and the problem each solves:
 - **RoPE** (rotary position embeddings) — relative position, extrapolates better than learned absolute
 - **RMSNorm** instead of LayerNorm — cheaper, no mean subtraction
 - **Pre-norm** instead of post-norm — trains stably at depth
 - **SwiGLU** feed-forward — better quality per parameter than ReLU MLP
 - **GQA / MQA** (grouped/multi-query attention) — shrinks the KV cache by 4–8x, which is why long context is affordable
 - **MoE** (mixture of experts) — more parameters at constant compute per token; brings routing and load-balancing problems
- CNNs: convolution, pooling, receptive field, residual blocks. Still the right tool for many vision problems
- Encoder-only (BERT-style, still the best choice for classification and embeddings), decoder-only (GPT-style), encoder-decoder (translation, speech)
- Diffusion models conceptually: forward noising, reverse denoising, classifier-free guidance, samplers
- State-space models (Mamba and successors) at AWARENESS level — linear-time sequence modelling, a real alternative for very long sequences

CHECKLIST | ESSENTIAL — training mechanics

- Loss functions and when each applies: cross-entropy, focal, contrastive (InfoNCE), triplet, cosine
- Optimisers: AdamW as the default, and what decoupled weight decay changed. Awareness of second-order and matrix-preconditioned optimisers (Shampoo, Muon) which have begun to show real wall-clock wins at scale
- Learning-rate schedules: warmup then cosine decay is the default; why warmup exists (early steps have unreliable second-moment estimates)
- Batch size and learning rate scaling; gradient accumulation to simulate a large batch
- Gradient clipping, and what a clipping event tells you
- Regularisation: dropout, weight decay, label smoothing, early stopping, data augmentation
- Initialisation and why it matters at depth
- Transfer learning and fine-tuning: which layers to freeze, discriminative learning rates

CHECKLIST | CORE — efficiency

- Gradient checkpointing: trade compute for activation memory, roughly $\sqrt{L}$ memory for one extra forward pass
- FlashAttention and why an algorithm that does *more* arithmetic is faster (it moves far less data)
- Quantisation-aware training and post-training quantisation
- Knowledge distillation from a large teacher to a small student
- Profiling to find whether you are compute-bound, memory-bound or input-bound — and the arithmetic to tell in advance

The debugging playbook

CHECKLIST | When the loss does not move

1. **Overfit a single batch.** If the model cannot drive the loss to zero on four examples, the bug is in the model or the loss, not the data or the schedule. This is the single most valuable diagnostic in deep learning.
2. **Check the learning rate** by sweeping over orders of magnitude. Wrong by 100x is the most common single cause.
3. **Check the label alignment.** Shuffled inputs relative to labels produces a loss that sits exactly at the entropy of the label distribution.
4. **Check the loss reduction and masking.** A mean over padded positions quietly scales your gradient by the padding fraction.
5. **Print shapes at every step.** A broadcast that should have been an error frequently is not.
6. **Check the data with your eyes.** Decode a batch back to text or images after the collate function, not before.

CHECKLIST | When the loss diverges or produces NaN

1. Lower the learning rate; add or lengthen warmup.
2. Add gradient clipping and log the pre-clip norm — a spike tells you when.
3. Check for `fp16` overflow; prefer `bf16`.
4. Look for division by a near-zero quantity: a variance, a norm, a softmax denominator. Add epsilon inside the square root, not outside.
5. Check for a corrupt example: sort the batch losses and inspect the worst.
6. Check the initialisation scale at depth.

COMMON PITFALL | The silent ones, which are worse

Train/eval mode. Forgetting `model.eval()` leaves dropout and batch-norm in training mode; evaluation numbers are wrong and nothing errors.

`zero_grad()` omitted. Gradients accumulate across steps. The model still trains, badly, and you will blame the learning rate.

Non-determinism blamed on the model. Unseeded dataloader workers, a non-deterministic kernel, or float reduction order. Fix the seeds before concluding anything about a change.

Evaluating on data the model saw. The most expensive silent bug; see Chapter 7.

How to tell you have it

PRACTICE | The task

Implement a decoder-only transformer from scratch in PyTorch — no `nn.Transformer`, no Hugging Face. Include RoPE, RMSNorm, pre-norm, SwiGLU and grouped-query attention. Then:

1. Predict every intermediate shape on paper before running it.
2. Overfit 64 tokens to near-zero loss.
3. Train it on a small corpus to a sensible perplexity.
4. Add gradient checkpointing and measure the memory and time change.
5. Add `torch.compile` and measure the speedup; count the graph breaks.
6. Deliberately introduce each of the six “loss does not move” bugs above and confirm you can identify each one from the symptom alone.

Step 6 is the one that builds the skill.

NOTE | Time to competence

8–12 weeks for ESSENTIAL at 12 hours a week, assuming the mathematics from Chapter 4 is solid. The from-scratch transformer is worth two weeks on its own and pays back permanently.

CHAPTER RECAP

- PyTorch is the default; know `torch.compile`, mixed precision (prefer `bf16`), and the profiler.
- Know the modern transformer components by the problem each solves: RoPE, RMSNorm, pre-norm, SwiGLU, GQA, MoE.
- AdamW, warmup then cosine, gradient clipping, and scaling the learning rate with the batch.
- Overfitting a single batch is the highest-value diagnostic; the six-step playbook resolves most “loss does not move” cases.
- The silent bugs — `eval()` mode, missing `zero_grad`, unseeded non-determinism, contaminated evaluation — cost more than the loud ones.

Training at Scale

When one GPU is not enough, and how to tell whether you need more

ADVANCED

PREREQS: Chapter 9

Scaling a training job is mostly a memory-accounting exercise followed by a communication-cost exercise. Most engineers will never pretrain a foundation model, and most will eventually need multi-GPU training for fine-tuning or for a large classical workload. This chapter gives you the arithmetic and the decision procedure.

WHAT YOU WILL LEARN

- Compute a training job's memory requirement before running it
- Choose among data, tensor, pipeline and expert parallelism from first principles
- Use FSDP or DeepSpeed correctly, and know what each stage shards
- Diagnose a scaling problem as compute, memory, communication or input bound

The arithmetic you do first

THE MATH | Memory for a P -parameter model, mixed precision with Adam

$$M \approx \underbrace{2P}_{\text{bf16 weights}} + \underbrace{2P}_{\text{grads}} + \underbrace{12P}_{\text{fp32 master} + m + v} = 16P \text{ bytes}$$

plus activations, which depend on batch size, sequence length and depth, and plus workspace.

A 7-billion-parameter model therefore needs about **112 GB** before a single activation is stored. That is why it does not fit on one 80 GB device and why sharding exists. A 1-billion-parameter model needs 16 GB and fits comfortably. Do this calculation *before* choosing a strategy; it usually decides it.

The capabilities

CHECKLIST | CORE — parallelism

- **Data parallel (DDP):** replicate the model, split the batch, all-reduce the gradients. Communication is $2(P-1)/P \times$ the parameter size per worker, independent of the worker count. Use when the model fits on one device
- **ZeRO / FSDP:** shard optimiser state (stage 1), then gradients (stage 2), then parameters (stage 3) across workers. Keeps the simplicity of data parallelism while removing its memory redundancy. This is the default for most people who need more than one GPU
- **Tensor parallel:** split each weight matrix across devices. Two all-reduces per layer, high volume — only viable over a fast interconnect, so it stays inside a node
- **Pipeline parallel:** split the layers into stages. Only boundary activations cross the link, so it tolerates slow interconnects. Costs a bubble of $(S-1)/(M+S-1)$, reduced by more micro-batches
- **Expert parallel:** for MoE, distribute experts and route tokens with an all-to-all
- **3D parallelism:** compose them so the heaviest traffic uses the fastest link — tensor within a node, pipeline across nodes, data across replicas

CHECKLIST | CORE — mechanics

- `torchrun` and process groups; NCCL, and reading an NCCL error
- Overlapping communication with the backward pass, and gradient bucketing
- Activation checkpointing configured per layer or per block
- Sharded, asynchronous checkpointing — gathering everything to rank zero is a serious and common mistake
- Mid-epoch resumption with correct dataloader state
- Cluster schedulers: Slurm in research settings, Kubernetes plus Ray or Kubeflow in industry
- Elastic and fault-tolerant training, because at scale hardware failure is routine rather than exceptional

CHECKLIST | CORE — diagnosing a scaling problem

Symptom	Likely cause	Check
GPU at 40%, CPU pegged	input pipeline	time the dataloader alone
GPU at 40%, CPU idle	I/O wait	storage throughput, shard size
Scales to 4 GPUs, not to 8	communication bound	measure all-reduce time
One rank always last	straggler, unequal shards	per-rank step timing
Out of memory at a larger batch	activations	checkpointing, micro-batching
Throughput falls over time	memory fragmentation, leak	monitor allocator stats

CHECKLIST | AWARENESS

- Scaling laws: the compute-optimal token-to-parameter relationship, and its practical inversion — for models that will serve many tokens, training a smaller model on more data is usually the better economic choice
- Muon, Shampoo and other matrix-preconditioned optimisers
- FP8 training and its scaling and stability considerations
- Megatron-LM and the mechanics of large pretraining stacks

KEY INSIGHT | The decision procedure, in order

1. Compute the memory requirement. If the model fits on one device with a usable batch size, use one device.
2. If it does not fit, first reduce: activation checkpointing, smaller micro-batch with accumulation, LoRA instead of full fine-tuning, 8-bit optimiser states.
3. If it still does not fit, use **FSDP/ZeRO**. This covers the large majority of real needs.
4. Only if the model does not fit even sharded, or you are compute-bound across many nodes, reach for tensor and pipeline parallelism.

Most teams reach step 4 without seriously attempting step 2, and pay for it in complexity and debugging time.

How to tell you have it

PRACTICE | The task

Fine-tune a model that does not fit on your GPU.

1. Compute the memory requirement on paper first and state which components dominate.
2. Get it running with LoRA on one device; record throughput.
3. Get it running with FSDP on two or more devices; record throughput and the scaling efficiency.
4. Add activation checkpointing and quantify the memory/time trade.
5. Profile and state whether you are compute, memory, communication or input bound — with a measurement, not a guess.
6. Kill a worker mid-run and resume from a checkpoint without losing more than one checkpoint interval.

Step 6 is the one that distinguishes people who have run real jobs.

COMMON PITFALL | Distributed training's silent failures

Different seeds per rank for the shuffle. Ranks see overlapping data; the effective epoch is wrong and nothing errors. Seed the shuffle identically and shard the result; seed augmentation differently.

Unequal shards. If the dataset does not divide by the world size, a rank finishes early and the collective hangs. Pad or drop explicitly.

Metrics computed on rank zero only. You report one shard's number.

Checkpointing through rank zero. It serialises the whole cluster and scales badly; shard the checkpoint.

NOTE | Time to competence

4–6 weeks for the CORE list given solid single-GPU training. The memory arithmetic is an afternoon and prevents most of the wasted effort.

CHAPTER RECAP

- Do the memory arithmetic first: roughly $16P$ bytes plus activations decides the strategy.
- Reduce before you distribute: checkpointing, accumulation, LoRA, 8-bit optimiser states.
- FSDP/ZeRO covers most real needs; tensor and pipeline parallelism are for when sharded data parallelism is genuinely insufficient.
- Ring all-reduce costs $2(P-1)/P \times N$ bytes per worker, independent of worker count; overlap it with the backward pass.
- Diagnose scaling problems by measurement: compute, memory, communication or input bound.
- The silent distributed failures are seeding, shard equality, rank-local metrics and centralised checkpointing.

PART V

Foundation Models

The layer that did not exist in this form five years ago and now accounts for most new AI engineering roles: retrieval, post-training, agents, and the evaluation that governs them.

-
- 11 Foundation Model Fundamentals
 - 12 Retrieval and RAG
 - 13 Post-Training: Fine-Tuning and Alignment
 - 14 Agents and Tool Use
 - 15 Evaluation

Foundation Model Fundamentals

What you must understand about the model you did not train

CORE

PREREQS: Chapter 9

Most AI engineers will never train a foundation model and will spend every day working with one. That requires a specific and slightly unusual body of knowledge: how tokenisation constrains you, what context windows really cost, how sampling parameters change behaviour, why structured output is a decoding problem, and where the model's failure modes come from.

WHAT YOU WILL LEARN

- Reason about tokenisation, context length and cost as a single budget
- Choose sampling parameters deliberately rather than by superstition
- Get reliable structured output and tool calls
- Know the standard failure modes and which are fixable by prompting

The capabilities

CHECKLIST | ESSENTIAL — tokenisation and context

- BPE and byte-level BPE; why the vocabulary is 50k–300k and what that implies
- Characters per token varies by language and content: roughly 4 for English prose, 3 for code, and often below 1.5 for languages poorly represented in the vocabulary. This is a cost and a capability difference, not a curiosity
- Why models are bad at character-level tasks (counting letters, reversing strings): they do not see characters
- Context windows: what “1M context” costs in latency and money, and that effective use degrades well before the stated limit
- The lost-in-the-middle effect: material at the start and end of a long context is used more reliably than material in the middle. Order your context accordingly
- Token budgeting: system prompt, retrieved context, conversation history, output reserve — and an explicit policy for what gets dropped first

CHECKLIST | ESSENTIAL — decoding and sampling

- Greedy, temperature, top-*k*, top-*p* (nucleus), min-*p*: what each truncation does and when it matters
- Temperature 0 is not deterministic in practice on GPU (batch-dependent float reduction order), and you should not build on the assumption
- Beam search is for tasks with a single right answer (translation, transcription) and is actively bad for open-ended generation
- Repetition, frequency and presence penalties, and their side effects
- Stop sequences and why they need to be chosen carefully
- Structured output: JSON mode, schema-constrained decoding, and why constraining the *format* is safe while constraining the *content* can force a confident wrong answer where the model would have declined

CHECKLIST | ESSENTIAL — prompting and context engineering

- The techniques that reliably help: clear role and task, explicit output format, few-shot examples chosen to cover the failure modes, decomposition into steps, giving the model permission to say it does not know
- Chain-of-thought, and that modern reasoning models do this internally — telling them to “think step by step” can now *hurt*
- Prompt structure: put stable content first so prefix caching can hit it; put the question last
- Delimiters and explicit sectioning of untrusted content
- Self-consistency (sample *n*, take the majority) when correctness matters more than cost
- The discipline of versioning prompts like code, with an eval per version

CHECKLIST | ESSENTIAL — tool use

- Function/tool calling: schema definition, the model’s selection behaviour, parallel calls, and handling malformed arguments
- MCP (Model Context Protocol) as the emerging standard for exposing tools and resources to models across vendors
- Designing tools the model can actually use: few tools, unambiguous names, descriptions written for the model rather than for a developer, errors that tell the model how to recover
- Idempotency and confirmation for anything with side effects

CHECKLIST | CORE — model selection and economics

- The tradeoff space: frontier API, open-weights hosted, open-weights self-hosted, small fine-tuned specialist
- Cost modelling in tokens: input versus output pricing, cached input discounts, and the fact that output tokens usually dominate cost while input tokens dominate latency-to-first-token
- Latency decomposition: time to first token (prefill, compute-bound, scales with prompt length) versus time per output token (decode, memory-bound, roughly constant)
- Routing: send easy traffic to a small model and hard traffic to a large one, with a classifier or a confidence signal
- Multimodal capability, and that a vision model's text performance is not the same as its base model's

CHECKLIST | AWARENESS

- Reasoning models and test-time compute: spending more tokens at inference to improve accuracy, and the fact that this is now a tunable rather than a fixed property
- Long-context architectures: sliding-window and sparse attention, and the serving consequences
- Model cards, evaluation claims and how to read a benchmark table sceptically

The standard failure modes

Failure	Cause	Fixable by prompting?
Confident fabrication	no grounding, no incentive to abstain	Partly: give a “not in the context” option and require citation
Ignores part of a long context	attention dilution, lost-in-the-middle	Partly: shorten, reorder, chunk and iterate
Format drift over a long output	no constraint	No — constrain decoding
Inconsistent across runs	sampling	Set temperature 0 <i>and</i> accept residual variation
Fails at counting or character edits	tokenisation	No — give it a tool
Sycophancy: agrees with a wrong premise	post-training incentives	Partly: ask it to evaluate the premise first
Degrades on long multi-turn chats	context growth, contradictory history	Summarise and externalise memory
Prompt injection from retrieved content	no trust boundary	No — this is an architecture problem, Chapter 19

KEY INSIGHT | The mental model that helps most

Treat the model as a very capable, very literal contractor with no memory between jobs, who will confidently do the wrong thing rather than ask a question, and who reads the middle of a long brief less carefully than the ends. Every effective prompting technique is a consequence of that model: be explicit, put the important thing where it will be read, give it permission to decline, and check its work.

How to tell you have it

PRACTICE | The task

Build an extraction service: unstructured documents in, a strict JSON schema out, with 95%+ schema validity and a measured field-level accuracy.

1. Establish a baseline with a plain prompt and measure it.
2. Add schema-constrained decoding and measure the change in validity and in accuracy — they can move in opposite directions.
3. Measure characters-per-token on your actual documents and compute cost per thousand documents for three models.
4. Measure time-to-first-token and time-per-output-token separately.
5. Add a routing rule sending short easy documents to a small model, and report the cost saving and the accuracy cost.

COMMON PITFALL | Prompt engineering without measurement

The dominant failure in this area is iterating on prompts by reading outputs and forming an impression. Impressions cannot detect a 3% regression, cannot be compared across people, and cannot be defended in a review. Build the evaluation harness (Chapter 15) before the third prompt revision, not after the thirtieth.

NOTE | Time to competence

3–4 weeks for ESSENTIAL if you are building daily. The economics and latency material is often skipped and is what makes the difference between a demo and a service.

CHAPTER RECAP

- Tokenisation determines cost, multilingual capability and the tasks the model cannot do; measure characters-per-token on your own data.
- Context is a budget with an explicit drop policy; position matters because the middle is read less carefully.
- Choose sampling parameters deliberately; beam search is wrong for open-ended generation; temperature 0 is not truly deterministic.
- Constrain format, not content; a forced schema can turn an honest refusal into a confident error.
- Design tools for the model: few, unambiguous, with recoverable errors. MCP is the emerging cross-vendor standard.
- Half the standard failure modes are architectural, not prompt-fixable.

Retrieval and RAG

A chain of decisions, each with its own failure mode and its own eval

CORE

PREREQS: Chapters 11, 5

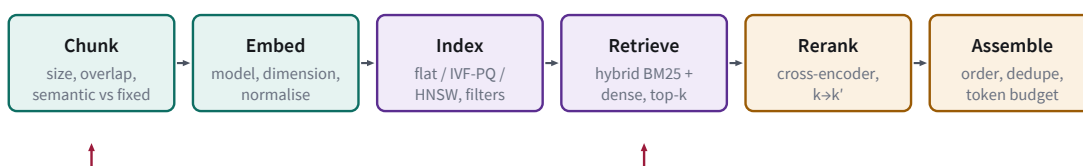
Retrieval-augmented generation is the most commonly built and most commonly mis-built pattern in applied AI. It is not one technique but a pipeline of six or seven decisions, each of which can silently destroy quality. The distinguishing skill is not building one; it is being able to say which stage is broken.

WHAT YOU WILL LEARN

- Build a retrieval pipeline and measure each stage independently
- Choose chunking, embedding, indexing and reranking strategies from the data
- Implement hybrid search and know when it beats dense retrieval alone
- Diagnose whether a bad answer is a retrieval failure or a generation failure

The pipeline

EACH BOX IS A TUNABLE WITH ITS OWN EVAL



Recall is lost at chunking and retrieval. Measure retrieval recall@k separately from answer quality:

if recall is 60%, no amount of prompt work on the generator will fix the answers — and teams spend months not knowing which half is broken.

Figure 12.1 A retrieval pipeline is a chain of tunable decisions. Recall lost at chunking or retrieval cannot be recovered downstream, which is why each stage needs its own measurement.

The capabilities

CHECKLIST | ESSENTIAL — chunking

- Fixed-size with overlap as a baseline; semantic and structural chunking (by heading, by section, by function) usually better
- Chunk size as a tradeoff: small chunks retrieve precisely and lose context; large chunks carry context and dilute the embedding
- Late chunking and contextual retrieval: embed with surrounding context or prepend a generated document-level summary to each chunk. This is one of the highest-yield cheap improvements available
- Preserving metadata (source, section, date, permissions) on every chunk — you will need it for filtering, citation and access control
- Handling tables, code and images, which fixed-size text chunking destroys

CHECKLIST | ESSENTIAL — embeddings and search

- Embedding model selection: dimension, context length, domain match, multilingual support, and cost. Benchmarks are a starting point, not an answer — evaluate on your own queries
- Normalisation, and why cosine and inner product coincide for unit vectors
- Asymmetric retrieval: queries and documents are different distributions; models with separate query/document prefixes exist for this reason
- Hybrid search: BM25 (lexical) plus dense (semantic), fused with reciprocal rank fusion. Lexical search is dramatically better at exact identifiers, codes, names and rare terms, which dense embeddings routinely miss
- Metadata filtering, and the fact that filtered vector search is genuinely hard (Chapter 33 of the companion volume)
- Reranking with a cross-encoder: retrieve 50–100, rerank to 5–10. Usually the largest single quality gain per unit of effort in the whole pipeline

CHECKLIST | ESSENTIAL — indexing

- Exact search is correct and fast below roughly a million vectors; do not build an index before you need one
- IVF, product quantisation, HNSW: what each trades, and the memory arithmetic
- Recall@*k* against a brute-force ground truth as the *only* honest measure of an index configuration
- Updates and deletions: HNSW deletes poorly; if your corpus churns, that constrains your architecture
- Vector database options: pgvector when you already have Postgres and the scale is modest, a dedicated store when it is not

CHECKLIST | CORE — query processing

- Query rewriting and expansion, and multi-query retrieval (generate several phrasings, retrieve for each, fuse)
- HyDE: generate a hypothetical answer and retrieve against that, which often matches the document distribution better than the question does
- Query routing: not every question needs retrieval, and a classifier that decides saves latency and reduces distraction
- Conversational retrieval: resolving “what about the second one?” into a standalone query before retrieving

CHECKLIST | CORE — generation and assembly

- Context ordering: most relevant at the edges rather than the middle
- Deduplicating retrieved chunks
- Citation and attribution, with spans if you can
- Instructing abstention when the context does not contain the answer, and measuring whether it complies
- Token budget management across system prompt, context, history and output

CHECKLIST | CORE — evaluation, which is the actual skill

Measure	What it tells you
Retrieval recall@k	did the right chunk get retrieved at all
Retrieval precision / MRR / NDCG	is the ranking any good
Context relevance	is the retrieved material actually about the question
Faithfulness / groundedness	is the answer supported by the retrieved context
Answer correctness	is the answer right, against a reference
Abstention accuracy	does it decline when it should
End-to-end latency and cost	can you afford it

KEY INSIGHT | The diagnostic that resolves most RAG problems in an afternoon

Measure retrieval recall separately from answer quality, on the same examples. There are only three cases, and each has a different fix:

- **Low recall, good answers when retrieval succeeds.** The problem is chunking, embedding or the index. Prompting will not help.
- **Good recall, bad answers.** The problem is generation: the prompt, the context ordering, the model, or the amount of distracting context.
- **Both bad.** Fix retrieval first; answer quality is bounded by it.

Teams routinely spend months tuning prompts against a retrieval problem, because they only ever looked at the final answer.

When RAG is the wrong answer

Situation	Better
The corpus fits comfortably in context	Put it in the prompt with prefix caching
The knowledge is stable and narrow	Fine-tune, or just write it in the system prompt
The question needs aggregation over many documents	Query a database; retrieval returns k chunks, not a GROUP BY
The question needs multi-hop reasoning over structure	A knowledge graph, or an agent with a search tool
You need guaranteed coverage	Retrieval is probabilistic; use a deterministic lookup

How to tell you have it

PRACTICE | The task

Build a retrieval system over a corpus you know well, then:

1. Construct 100 question/answer pairs with the *correct chunk* labelled — this is the work, and it is what makes everything else measurable.
2. Report recall@5 and recall@20 for: fixed chunking, semantic chunking, dense only, hybrid, and hybrid plus reranking. Five numbers.
3. Report faithfulness and answer correctness for the best two configurations.
4. Find three questions the system answers wrongly and classify each as a retrieval or a generation failure, with evidence.
5. Measure end-to-end p50 and p95 latency and cost per query.

COMMON PITFALL | The five RAG mistakes

1. **No retrieval metric.** Only end-to-end quality is measured, so nobody knows which half is broken.
2. **Dense-only search.** Exact identifiers, error codes and rare names fail; BM25 would have found them.
3. **No reranker.** The cheapest large quality gain, routinely skipped.
4. **Chunks without context.** A chunk saying “it increased by 12%” is useless without knowing what “it” is.
5. **Building an ANN index at 50,000 documents.** Brute force is exact, faster, and has no tuning.

NOTE | Time to competence

4–6 weeks to build and *evaluate* a real pipeline. Building one takes a weekend; the evaluation is the skill and the remainder of the time.

CHAPTER RECAP

- RAG is a chain of decisions — chunk, embed, index, retrieve, rerank, assemble — each with its own failure mode and its own metric.
- Hybrid lexical plus dense search, and a cross-encoder reranker, are the two highest-yield additions to a naive pipeline.
- Contextual or late chunking fixes the “chunk without context” failure cheaply.
- Do not build an ANN index below roughly a million vectors.
- Measure retrieval recall separately from answer quality; that one split resolves most RAG debugging.
- RAG is wrong for aggregation questions, for stable narrow knowledge, and when guaranteed coverage is required.

Post-Training: Fine-Tuning and Alignment

The rungs of the adaptation ladder, and why most teams climb too far

ADVANCED

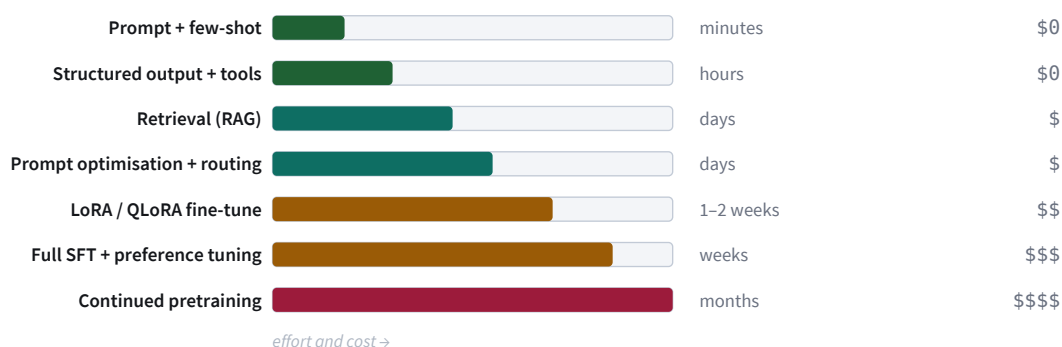
PREREQS: Chapters 9, 11

Fine-tuning has become accessible enough that teams reach for it far too early, and powerful enough that when it is the right tool it is decisive. This chapter covers the full adaptation ladder from prompting to preference optimisation, with an emphasis on the decision of where to stop.

WHAT YOU WILL LEARN

- Choose the cheapest intervention that could solve the problem
- Run a LoRA or QLoRA fine-tune correctly, including the data work
- Understand SFT, reward models, and the modern preference-optimisation family
- Evaluate a fine-tune honestly, including what it broke

The ladder



Climb one rung at a time and measure. The most common expensive mistake in applied AI is fine-tuning a model that a better prompt and a retrieval step would have fixed for nothing.

Figure 13.1 Each rung costs roughly an order of magnitude more effort than the one below it. Climb one at a time and measure. The single most expensive mistake in applied AI is fine-tuning around a problem that better context would have solved.

When fine-tuning is actually right

Fine-tune when	Do not fine-tune when
You need a consistent format, tone or style	You need the model to know new facts
You need a narrow task done cheaply by a small model	The knowledge changes frequently
You need lower latency or cost at fixed quality	You have fewer than a few hundred good examples
The task is hard to specify but easy to demonstrate	A better prompt or retrieval step is untried
You need to distil a large model's behaviour into a small one	You cannot evaluate the result

The capabilities

CHECKLIST | CORE — supervised fine-tuning

- **Data format:** instruction/response pairs, chat templates, and getting the template exactly right for the base model (a mismatched template silently destroys quality)
- **Loss masking:** train on the response tokens only, not on the prompt
- **Data quality over quantity** — a thousand carefully curated examples routinely beat a hundred thousand scraped ones
- **Packing sequences** to avoid padding waste; attention masking across packed document boundaries
- **Hyperparameters:** a much lower learning rate than pretraining (10^{-5} to 10^{-4} for full, 10^{-4} to 10^{-3} for LoRA), one to three epochs, and early stopping on a held-out set
- **Catastrophic forgetting:** measure general capability before and after, not only the target task

CHECKLIST | CORE — parameter-efficient fine-tuning

- **LoRA:** freeze the base weights and learn a low-rank update $\Delta W = BA$. Rank 8–64 covers most cases; also choose which modules to adapt (attention projections at minimum, often the MLP too)
- **QLoRA:** quantise the frozen base to 4-bit and train LoRA adapters on top, which brings large models within reach of a single GPU
- The alpha/rank scaling relationship, and why people get it wrong
- Merging adapters back into base weights for serving, and serving multiple adapters against one base model
- Awareness of the wider PEFT family: prefix tuning, prompt tuning, IA³, DoRA

CHECKLIST | CORE — preference optimisation

- Why SFT alone is insufficient: it teaches the model to imitate, not to prefer, and cannot express “this answer is better than that one”
- **RLHF**: train a reward model on pairwise preferences, then optimise the policy with PPO against it, with a KL penalty to the reference model. Powerful, complex, and unstable
- **DPO**: optimise directly on preference pairs with a closed-form objective and no separate reward model. Far simpler, now the default starting point for most teams
- The wider family and what each changes: IPO (regularisation), KTO (works from unpaired binary feedback), ORPO (folds preference into SFT), SimPO
- **GRPO** and group-relative methods: sample a group of completions, score them, and use the within-group relative advantage. Central to training reasoning models with verifiable rewards
- **RLVR** (reinforcement learning from verifiable rewards): when correctness is checkable — maths, code that runs, format that parses — the reward is a program rather than a model, which removes reward hacking at its source
- Reward hacking, and why a held-out qualitative review is not optional

CHECKLIST | CORE — evaluation of a fine-tune

- A held-out set from the same distribution, built *before* training
- General capability benchmarks before and after, to measure what you broke
- Format and instruction-following compliance
- Safety behaviour, which fine-tuning frequently degrades even when the training data is benign
- A/B comparison against the pre-fine-tune model on real traffic where possible

CHECKLIST | AWARENESS

- Continued pretraining for a new domain or language
- Model merging and model soups
- Distillation: training a small student on a large teacher’s outputs or logits — often the highest-value production technique in this chapter
- Quantisation-aware fine-tuning

KEY INSIGHT | The economic case that actually justifies fine-tuning

It is rarely “the big model cannot do this”. It is usually: the big model can do it, at X per million tokens and Y milliseconds, and a fine-tuned 3B model can do it at $X/30$ and $Y/5$. At sufficient volume that difference funds the entire project. Frame the decision as an economics question with measured numbers, not as a capability question, and it becomes much easier to answer.

How to tell you have it

PRACTICE | The task

Pick a narrow task where you can measure correctness automatically.

1. Establish a prompted baseline on a frontier model and on a small model. Record quality, latency and cost.
2. Add retrieval or better context. Measure again. Frequently you stop here.
3. Build 500–2,000 high-quality training examples. This is most of the work.
4. LoRA fine-tune the small model. Measure quality, latency and cost.
5. Measure what you broke: general benchmarks, instruction-following, safety behaviour.
6. Compute the break-even volume at which the fine-tune pays for itself, including your time.
7. Write down, honestly, whether step 2 was already good enough.

COMMON PITFALL | The five fine-tuning failures

1. **Wrong chat template.** The most common silent failure; quality collapses for no visible reason.
2. **Training on the prompt tokens.** Loss masking omitted; the model learns to generate prompts.
3. **Fine-tuning for knowledge.** Facts learned this way are unreliable and unpatchable. Use retrieval.
4. **No before/after on general capability.** You fixed one task and degraded ten.
5. **Too few, too low-quality examples.** A few hundred excellent examples beat tens of thousands of mediocre ones, and most teams have the ratio backwards.

NOTE | Time to competence

6–8 weeks for SFT and LoRA including the data work; another **4–6 weeks** for preference optimisation. The data curation is the majority of the time in both cases, which is itself the lesson.

CHAPTER RECAP

- Climb the adaptation ladder one rung at a time: prompt, structure, retrieve, optimise, LoRA, full SFT, preference tuning.
- Fine-tune for behaviour, not for knowledge; facts belong in retrieval.
- LoRA and QLoRA cover most real needs; get the chat template and the loss masking right or nothing else matters.
- DPO is the practical default for preference optimisation; GRPO and verifiable rewards are how reasoning behaviour is trained.
- Always measure what the fine-tune broke, not only what it improved.
- The justification is usually economic — cost and latency at volume — not a capability the big model lacks.

Agents and Tool Use

A while-loop with a budget, and the four places it goes wrong

ADVANCED

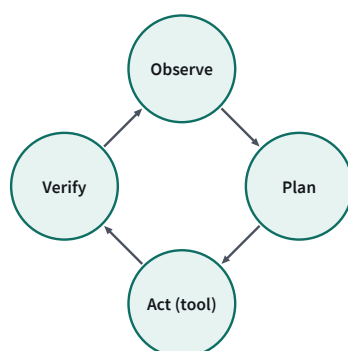
PREREQS: Chapters 11, 12

“Agent” covers everything from a single tool call to an autonomous multi-step system. The engineering content is smaller and more mundane than the discourse suggests: tool design, state management, budgets, verification and failure handling. The hard parts are reliability and evaluation, not architecture.

WHAT YOU WILL LEARN

- Build a reliable tool-using loop with explicit budgets and verification
- Design tools a model can actually use correctly
- Manage context and memory across long-running tasks
- Evaluate an agent, which is substantially harder than evaluating a single response

The loop, and its failure modes



FAILS LIKE THIS

Loops forever
Tool call malformed
Wrong tool chosen
Context overflows
Silent wrong answer

FIX

hard step budget + a cost ceiling
constrained decoding against the schema
fewer, better-described tools; eval per tool
summarise or externalise memory
a verifier step, or human sign-off

An agent is a while-loop with a budget. Most production “agents” are three tools and a good verifier.

Figure 14.1 An agent is a loop with a budget. The right-hand column is where the engineering actually goes.

The capabilities

CHECKLIST | CORE — the loop

- The basic pattern: observe, plan, act, verify, repeat until done or out of budget
- Hard budgets on steps, wall-clock time, tokens and money — all four, enforced, with a defined behaviour on exhaustion
- Error handling that feeds the error back to the model in a form it can act on (“the file does not exist; here are the files that do”) rather than a stack trace
- Termination conditions and detecting loops (repeated identical actions)
- Structured state so that a run can be inspected, resumed and replayed
- Streaming progress to the user, because a silent 40-second agent feels broken

CHECKLIST | CORE — tool design

- Few tools with unambiguous boundaries. Model performance degrades noticeably as the tool count grows; twenty overlapping tools is a design smell
- Descriptions written for the model: what it does, when to use it, when *not* to, what the arguments mean, what errors look like
- Schema-constrained arguments, validated before execution
- Idempotency, so a retry is safe
- Confirmation gates for irreversible or costly actions
- Sandboxing anything that executes code or touches a network. This is a security boundary, not a nicety (Chapter 19)
- MCP for exposing tools in a standard, reusable way across clients

CHECKLIST | CORE — context and memory

- Context is the scarce resource. Strategies, roughly in order of preference: summarise old turns, externalise state to a file or database the agent can read back, retrieve selectively rather than carrying everything, and start a fresh context with a handoff note
- Distinguish working memory (this task), episodic memory (what happened before) and semantic memory (learned facts); they have different storage and retrieval needs
- Compaction: when and what to summarise, and keeping the summary faithful
- Passing state between sub-agents explicitly rather than hoping it is in the transcript

CHECKLIST | CORE — architecture patterns

Pattern	Use when
Single tool call	The task is one lookup or one action. Most tasks.
ReAct loop (think, act, observe)	Multi-step, tools available, moderate ambiguity
Plan-then-execute	The plan is worth reviewing before acting
Reflection / self-critique	Quality matters more than latency; pair with a verifier
Router	Several specialised paths; classify then dispatch
Orchestrator with sub-agents	Genuinely separable sub-tasks, each with its own context
Human in the loop	Irreversible actions, or accuracy below the required bar

CHECKLIST | CORE — evaluation

- Trajectory evaluation, not just final-answer evaluation: did it take a sensible path, call the right tools, avoid unnecessary steps
- Per-tool accuracy: was each call well formed and appropriate
- Cost and step distributions, not just means — the tail is what hurts
- Reproducible runs: fixed seeds, recorded tool responses, replayable transcripts
- Failure taxonomy: classify every failure into a small set of causes and track the mix over time. This is how you know what to fix next

CHECKLIST | AWARENESS

- Multi-agent debate, role-play and voting schemes — occasionally useful, frequently an expensive way to get a small gain
- Computer use and browser automation
- Long-horizon autonomy and its safety implications

KEY INSIGHT | The engineering opinion that holds up

Most production “agents” are: three or four well-described tools, a loop with a step budget, structured output, a verifier, and extensive logging. The elaborate multi-agent architectures that appear in diagrams are usually slower, more expensive and harder to debug than a single well-instrumented loop, and they very rarely win in an evaluation. Start with a single loop and three tools. Add complexity only when a measured failure mode demands it.

How to tell you have it

PRACTICE | The task

Build an agent that answers questions requiring several steps over a real data source — a database and a document store, say.

1. Implement the loop with explicit step, token and cost budgets.
2. Give it three tools with schemas and validation.
3. Build 50 evaluation tasks with known correct answers *and* known correct tool-use paths.
4. Report: success rate, mean and p95 steps, mean and p95 cost, and the failure mix by category.
5. Add a verification step and re-measure. Report whether it paid for itself.
6. Deliberately break each tool (timeout, malformed response, empty result) and confirm the agent recovers or fails cleanly rather than looping.

Step 6 is what separates a demo from a service.

COMMON PITFALL | The four agent failures

1. **No budget.** A loop with no step or cost ceiling will eventually find an input that makes it run until somebody notices the bill.
2. **Too many tools.** Selection accuracy falls; consolidate.
3. **Evaluating only the final answer.** You cannot improve what you cannot localise; evaluate the trajectory.
4. **No sandbox.** An agent with shell access and a prompt-injectable context is a remote code execution vulnerability with extra steps.

NOTE | Time to competence

4–6 weeks to build reliable single-agent systems. Evaluation is the long pole and the differentiating skill.

CHAPTER RECAP

- An agent is a loop with a budget; enforce steps, time, tokens and money.
- Design few tools, described for the model, with validated schemas, idempotency, confirmation gates and a sandbox.
- Context is the scarce resource: summarise, externalise, retrieve selectively, hand off.
- Evaluate trajectories and per-tool accuracy, track a failure taxonomy, and look at tails rather than means.
- Start with one loop and three tools; add architecture only when a measured failure demands it.

Evaluation

The scarcest skill in applied AI, and the most learnable

CORE

PREREQS: Chapters 4, 11

Almost every team building on foundation models is bottlenecked by the same thing: they cannot tell whether a change made the system better. Evaluation is unglamorous, it is rarely taught, and engineers who can build an honest harness are scarce out of all proportion to how hard the skill actually is. If you read one chapter of this map, read this one.

WHAT YOU WILL LEARN

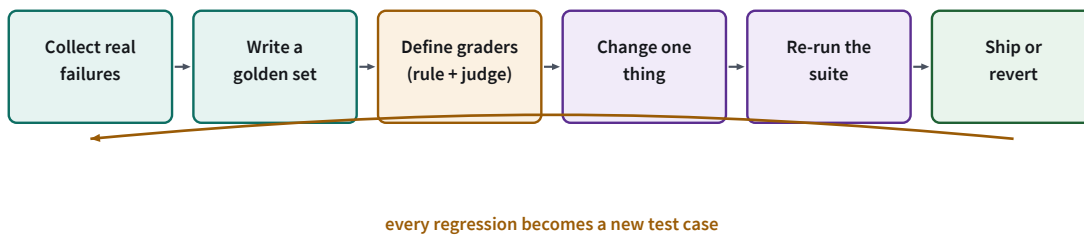
- Build a golden set that is representative, adversarial and maintained
- Write graders — rule-based, model-based and human — and validate them
- Make claims with statistical honesty, including at small sample sizes
- Run eval-driven development as a working loop rather than an aspiration
- Monitor quality in production, not only before deployment

Why this is the bottleneck

A team without an evaluation harness can still ship. What they cannot do is *improve reliably*. Every change becomes a judgement call, regressions are discovered by users, and after three months nobody can say which of the forty prompt revisions helped. The symptom is a system that plateaus early and then oscillates.

The reason this skill is scarce is that it feels like overhead. It is not: it is the thing that converts effort into progress.

The loop



Teams that do this ship steadily. Teams that judge output by eye plateau within a month and cannot tell whether a change helped.

Figure 15.1 Eval-driven development. The return arrow is the part that compounds: every production failure becomes a permanent test case, so the same regression cannot happen twice.

The capabilities

CHECKLIST | ESSENTIAL — datasets

- Build a **golden set**: 100–1,000 examples with known-good outputs or known-correct judgements. Start at 50 if you must; 50 is infinitely better than zero
- Make it representative of real traffic, which means sampling from real traffic rather than inventing examples
- Include the hard cases deliberately: ambiguous inputs, adversarial inputs, out-of-scope questions that should be declined, edge cases, and examples where the right answer is “I do not know”
- Maintain it: every production failure becomes a test case, permanently
- Version it, and treat a change to the eval set as a change requiring review — otherwise you can improve your score by weakening your test
- Keep a second held-out set you do not iterate against, to catch overfitting to the first one

CHECKLIST | ESSENTIAL — graders

- **Rule-based** where possible: exact match, regex, schema validity, code that runs, numerical tolerance, latency threshold. Cheap, deterministic, and the strongest grader when the task permits it
- **Reference-based**: ROUGE, BLEU, BERTScore, embedding similarity. Weak proxies for quality; use them as regression alarms, not as quality measures
- **LLM-as-judge**: a model scores the output against a rubric. Now the standard for open-ended tasks, with real caveats:
 - Write a *rubric*, not “rate 1–10”. Specific criteria with definitions
 - Pairwise comparison is more reliable than absolute scoring
 - Randomise position: judges have a position bias
 - Known biases: length, verbosity, self-preference toward the judge’s own model family, and sensitivity to formatting
 - **Validate the judge against human labels**. Measure agreement. A judge you have not validated is a number generator
 - Use a different model as judge than the one under test where you can
- **Human evaluation** as ground truth: clear guidelines, multiple annotators, measured inter-annotator agreement, adjudication

CHECKLIST | ESSENTIAL — statistics

- Report a confidence interval with every number. Bootstrap is easy and distribution-free
- Paired comparisons where possible: run both systems on the same examples and test the *difference*, which has far lower variance than comparing two independent means
- Know how many examples you need. To detect a 5-point difference in a proportion near 0.8 with reasonable power you need a few hundred paired examples; for 1 point you need thousands. Compute this before claiming
- Correct for multiple comparisons: evaluating twenty prompts on one set will produce a winner by chance
- Watch for a non-stationary judge: model updates change the grader, so pin versions and re-baseline when you cannot

CHECKLIST | CORE — what to measure

Layer	Measure
Component	retrieval recall@k, tool-call validity, schema compliance, classifier accuracy
Output	correctness, faithfulness, relevance, completeness, tone, format
Behavior	refusal appropriateness, abstention, instruction-following, safety
Operational	p50/p95 latency, cost per request, error rate, throughput
Business	task completion, escalation rate, user edits, thumbs-down rate

The vertical structure matters: an end-to-end number tells you *that* something is wrong, and component metrics tell you *what*.

CHECKLIST | CORE — in production

- Online evaluation: sample real traffic, grade it continuously, alert on drift in the grade distribution
- Implicit signals: user edits, retries, abandonment, escalation to a human, copy events. Often more honest than explicit thumbs
- Explicit feedback, with the knowledge that it is heavily biased toward extremes
- A/B testing with proper randomisation and sequential-testing discipline
- Shadow deployment: run the new system alongside, grade both, ship nothing
- Canary releases with automatic rollback on a metric regression
- Regression suites in CI: the eval runs on every change to prompt, model or retrieval configuration, and blocks the merge if it drops

CHECKLIST | CORE — error analysis

- Read the failures. Fifty of them, by hand, personally. There is no substitute and there is no shortcut
- Cluster them into a taxonomy; count the clusters; fix the biggest one
- Re-cluster monthly, because the mix changes as you fix things
- Keep the taxonomy in the repository so the team shares a vocabulary for what is going wrong

Tools

Job	Options
Harness and tracing	Braintrust, LangSmith, W&B Weave, Phoenix/Arize, OpenTelemetry-based tracing
Open-source frameworks	<code>promptfoo</code> , <code>deepeval</code> , <code>ragas</code> (RAG-specific), <code>inspect</code>

Job	Options
Academic benchmarks	MMLU, GPQA, HumanEval, SWE-bench, and their successors
Your own	A CSV, a grader function and a script. Genuinely fine to start

WATCH OUT | Public benchmarks measure the wrong thing for you

MMLU and its relatives are useful for comparing base models and useless for telling you whether your system works. They are contaminated to an unknown degree, they do not resemble your traffic, and improving on them does not improve your product. Use them to choose a starting model; use your own golden set for everything after that.

How to tell you have it

PRACTICE | The task

Take any LLM feature, yours or somebody else's.

1. Sample 100 real inputs. If there is no real traffic, write 100 inputs and label where they came from.
2. Produce reference outputs or judgements for all 100.
3. Write three graders: one rule-based, one LLM-judge with a written rubric, one human protocol.
4. **Validate the judge:** have a human label 50 of them and report agreement with the judge. If agreement is poor, fix the rubric before trusting any number it produces.
5. Establish the baseline with a bootstrap confidence interval.
6. Make one change. Re-run. State whether the difference is significant, using a paired test.
7. Wire it into CI so the suite runs on every prompt change.
8. Read 20 failures by hand and write the taxonomy.

Step 4 is the one nearly everybody skips, and it is the one that makes the rest of the numbers mean anything.

COMMON PITFALL | The six evaluation failures

1. **Vibes.** Reading outputs and forming an impression. Cannot detect a 3% regression, cannot be compared across people.
2. **An unvalidated judge.** A number with no established relationship to quality.
3. **Evaluating on the set you tuned against.** Scores improve, quality does not.
4. **No confidence intervals.** Noise gets shipped as improvement.
5. **Only end-to-end metrics.** You know something is wrong, not what.
6. **A stale eval set.** It reflects last quarter's traffic and last quarter's failures.

NOTE | Time to competence

3–4 weeks to build a good harness for one system, and it is the highest return on investment in this entire map. Most of the skill is judgement about what to measure, which you acquire by reading failures.

CHAPTER RECAP

- Teams are bottlenecked by the inability to tell whether a change helped; evaluation is the fix and it is learnable in weeks.
- Build a versioned golden set from real traffic, including hard cases and cases that should be declined; grow it from every production failure.
- Prefer rule-based graders; use LLM judges with written rubrics, pairwise comparison, position randomisation, and validation against human labels.
- Report confidence intervals, use paired tests, and compute the sample size before claiming an improvement.
- Measure components as well as end-to-end, or you cannot localise a regression.
- Public benchmarks choose a base model; only your own set measures your system.
- Read fifty failures by hand and maintain a taxonomy. There is no substitute.

PART VI

Production

Getting a model to work is a fraction of the job. Serving it cheaply, observing it honestly, and keeping it correct after the world moves is the rest.

-
- 16 Inference and Serving
 - 17 MLOps and Production Practice
 - 18 Hardware and Systems

Inference and Serving

Where quality meets the bill

ADVANCED

PREREQS: Chapters 11, 5

Serving a model well is now a discipline in its own right, and it is where an enormous amount of money is either saved or wasted. The governing fact is that token generation is memory-bandwidth bound, not compute bound, and almost every serving optimisation follows from that single observation.

WHAT YOU WILL LEARN

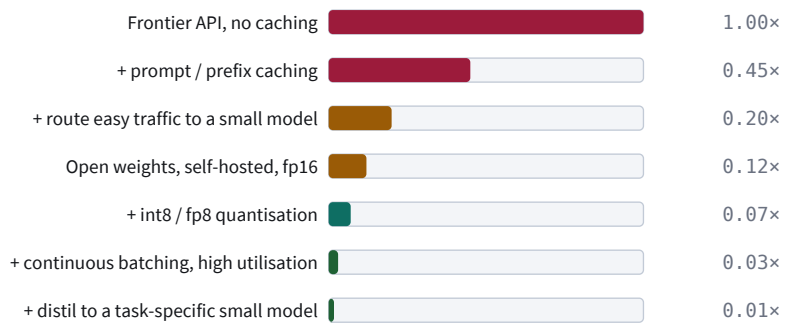
- Explain why decoding is bandwidth-bound and what follows from it
- Compute the memory budget for a serving deployment
- Use batching, quantisation, caching and speculative decoding appropriately
- Choose between an API and self-hosting on defensible economics

The governing fact

Generating one token requires reading every weight the model uses from memory. For a 7-billion-parameter model in fp16 that is 14 GB read per token. The arithmetic performed is about 14 GFLOPs. On a device with 2 TB/s of bandwidth and 500 TFLOP/s of compute, the read takes 7 ms and the arithmetic takes 0.03 ms. **Decoding is roughly two hundred times memory-bound.**

Every serving technique below is an attack on that ratio: read fewer bytes per token, or amortise the read over more tokens.

The cost ladder



Relative cost per million served tokens, same task, same quality bar. Two orders of magnitude sit between the top and the bottom row — and none of it is model quality work.

Figure 16.1 Relative cost per million served tokens for the same task at the same quality bar. Two orders of magnitude separate the top and bottom rows, and none of it is model-quality work.

The capabilities

CHECKLIST | CORE — the fundamentals

- Prefill versus decode: prefill processes the prompt in parallel and is compute-bound; decode generates one token at a time and is bandwidth-bound. They have opposite bottlenecks, which is why some systems run them on separate hardware
- Time to first token (dominated by prefill and queueing) versus time per output token (dominated by bandwidth). Users perceive them differently, so measure and budget them separately
- **Continuous batching:** the single largest throughput lever. Because decode is bandwidth-bound, serving 32 sequences costs little more than serving one — the weights are read once either way
- **KV cache** memory: $4 \cdot B \cdot S \cdot L \cdot H_{kv} \cdot d_h$ bytes in fp16. Compute it for your model and context length; it usually bounds your batch size and therefore your throughput
- **Paged attention:** store the KV cache in fixed-size blocks with a per-request block table, eliminating fragmentation and enabling sharing
- **Prefix caching:** shared system prompts and multi-turn history are recomputed for nothing otherwise; hit rates of 50–90% are typical on chat traffic

CHECKLIST | CORE — making it cheaper

- **Quantisation:** int8 and fp8 weights halve or quarter the bytes read per token, which on a bandwidth-bound workload is close to a proportional speedup. Know the families — GPTQ, AWQ, GGUF for CPU/edge — and always measure the quality cost on your own evaluation set rather than trusting a published number
- **KV cache quantisation**, which extends context or batch at a small quality cost
- **Speculative decoding:** a small draft model proposes several tokens and the large model verifies them in one pass. Provably preserves the output distribution, so it is safe to enable by default. Variants use n -gram lookup from the prompt or extra prediction heads instead of a draft model
- **Structured and constrained decoding** for guaranteed-valid output, at negligible runtime cost
- **Distillation** to a task-specific small model, which is the largest saving available when the task is narrow
- **Routing:** classify the request and send easy traffic to a small model
- **Semantic caching** of whole responses, with care about correctness

CHECKLIST | CORE — operations

- Serving engines: vLLM and SGLang are the common open-source choices; TensorRT-LLM where you are on NVIDIA and want the last ten per cent; llama.cpp for CPU and edge
- Autoscaling on GPUs, where cold start is minutes and spot interruption is routine
- Load balancing with session affinity so a follow-up turn lands on the machine holding its prefix cache
- Queueing and admission control; shedding load gracefully under overload
- Timeouts, retries with backoff and jitter, circuit breakers, and a fallback model
- Streaming responses, because perceived latency is mostly time to first token
- Observability: tokens per second, queue depth, cache hit rate, GPU utilisation, cost per request

CHECKLIST | CORE — the build-or-buy decision

Favours an API	Favours self-hosting
Low or spiky volume	High steady volume
You need frontier quality	A small model suffices
Small team, no GPU operations experience	You have platform engineering
Requirements still changing	Requirements stable
No data-residency constraint	Data cannot leave your boundary
Latency budget is generous	You need tight tail latency control

The honest break-even is usually higher than people expect: below roughly a sustained few hundred million tokens a month, an API is normally cheaper once engineering time is counted.

CHECKLIST | AWARENESS

- Disaggregated serving: prefill and decode on separate hardware pools
- Multi-LoRA serving: many adapters against one base model
- Edge and on-device inference, and the quantisation that makes it possible
- Compilation stacks: TensorRT, torch.compile with inductor, ONNX Runtime

How to tell you have it

PRACTICE | The task

Serve an open-weights model yourself.

1. Compute the KV cache size and the maximum batch size on paper first.
2. Deploy with vLLM or SGLang. Measure tokens per second at batch sizes 1, 8, 32 and 64, and plot throughput against latency.
3. Enable prefix caching; measure the hit rate and the time-to-first-token change on a workload with a shared system prompt.
4. Quantise to int8. Measure the throughput gain *and* the quality change on your evaluation set. Report both.
5. Enable speculative decoding; measure the acceptance rate and the net speedup.
6. Compute cost per million tokens for your deployment and compare with an API at the same quality. State the break-even volume.

COMMON PITFALL | The four serving mistakes

1. **Optimising the model when the bottleneck is elsewhere.** Measure queueing, prefill and decode separately before touching anything.
2. **Batch size one.** The most common cause of terrible economics; it wastes almost all of the hardware.
3. **Quantising without an eval.** The quality cost is task-dependent and sometimes large; a published benchmark number is not evidence for your task.
4. **Self-hosting at low volume.** GPU idle time and engineering hours dominate; the spreadsheet usually says otherwise than the instinct does.

NOTE | Time to competence

4–6 weeks for the CORE list with access to a GPU. The memory arithmetic is an afternoon and prevents most of the wasted effort.

CHAPTER RECAP

- Decoding is memory-bandwidth bound; every serving optimisation reduces bytes read per token or amortises the read over more tokens.
- Continuous batching is the largest throughput lever; KV cache size bounds the batch and therefore the throughput.
- Paged attention removes fragmentation; prefix caching removes recomputation of shared prompts.
- Quantisation buys close to proportional speedup on this workload; always measure the quality cost yourself.
- Speculative decoding preserves the output distribution exactly.
- Prefill and decode have opposite bottlenecks; measure and budget TTFT and TPOT separately.
- The API-versus-self-hosting break-even is higher than instinct suggests.

MLOps and Production Practice

Everything between a working model and a system you can operate

CORE

PREREQS: Chapters 3, 6

MLOps is what happens when software operations meets a system whose behaviour depends on data that changes. The additional problems over ordinary software are reproducibility, drift, and the fact that a broken model produces plausible output rather than an error. This chapter covers the practices that address each.

WHAT YOU WILL LEARN

- Make an experiment reproducible by construction rather than by discipline
- Version models, data and code together and roll back all three
- Deploy safely with shadow, canary and automatic rollback
- Monitor the things that actually detect a degrading system

The capabilities

CHECKLIST | ESSENTIAL — experiment tracking and reproducibility

- Track every run: code commit, data version, config, environment, hyperparameters, metrics, artefacts. W&B and MLflow are the common choices
- Seed everything and record the seeds; know what remains non-deterministic on a GPU and decide whether you care
- Data versioning: DVC, LakeFS, Delta/Iceberg time travel, or content-hashed snapshots. “Trained on the data as of last Tuesday” must be a resolvable statement
- The reproducibility test: can you regenerate a model from three months ago and get the same metric? If not, you cannot debug a regression

CHECKLIST | ESSENTIAL — CI/CD for ML

- Tests that run on every change: unit, data validation, a single-batch overfit test, a small end-to-end training smoke test, and the evaluation suite
- Model registry with stages (staging, production, archived) and a recorded lineage back to data and code
- Automated retraining pipelines where the data changes; a manual gate where the stakes are high
- Rollback that restores the model *and* the preprocessing *and* the config, because rolling back one of the three is how skew is introduced

CHECKLIST | ESSENTIAL — deployment patterns

Pattern	What it gives you
Shadow	Run the new model on real traffic, serve the old one, compare offline. Zero risk
Canary	Route 1–5%, watch metrics, expand or roll back automatically
Blue/green	Instant switch and instant rollback, at double the capacity
A/B	Measure the <i>business</i> effect, not just the model metric
Feature flag	Turn a model off without a deploy. Non-negotiable for LLM features

CHECKLIST | CORE — monitoring

- **Operational:** latency percentiles, error rate, throughput, saturation, cost per request
- **Data:** schema violations, null rates, range violations, freshness, volume
- **Drift:** input distribution (PSI, KS, or a simple binned divergence), prediction distribution, and — when labels eventually arrive — actual performance
- **Quality:** for LLM systems, continuous sampled grading against the rubric from Chapter 15
- **Business:** the metric the system exists to move
- Alert on the last two; dashboard the rest. Alerting on input drift produces noise, because features move constantly without performance changing

CHECKLIST | CORE — the things that make it operable

- Runbooks: what to do when the alert fires, written before it fires
- On-call for ML systems, and accepting that “the model is worse” is a real incident class
- Postmortems that reach the data or process cause rather than stopping at “the model degraded”
- Cost attribution per feature, per team, per request. Without it, cost optimisation has no target
- Documentation: model cards, data sheets, decision records

CHECKLIST | CORE — LLM-specific operations

- Prompt versioning as deployable artefacts, with an eval gate on change
- Model version pinning, and a plan for provider deprecations — they happen with months of notice and break behaviour subtly
- Full request/response tracing with token counts and cost per span
- Semantic caching, with an invalidation story
- Spend limits and per-tenant quotas enforced in code
- Regression suites in CI on every prompt, model or retrieval change

CHECKLIST | AWARENESS

- Kubernetes for ML workloads: GPU scheduling, node selectors, operators
- Ray for distributed Python; KServe or BentoML for model serving
- Terraform and infrastructure as code
- Multi-region, data residency and failover

KEY INSIGHT | The difference from ordinary software operations

A conventional service fails loudly: a 500, a timeout, a stack trace. A machine learning system fails *quietly and plausibly* — it returns a confident, well-formatted, wrong answer, and every conventional monitor stays green. This single difference justifies everything in this chapter that looks like overhead: continuous quality sampling, drift monitoring, shadow deployment, and treating “the output got worse” as a pageable incident.

How to tell you have it

PRACTICE | The task

Take a model you have trained and put it into a state you could hand to an on-call engineer.

1. Reproduce a three-month-old result from the tracked run. If you cannot, fix that first.
2. Register the model with a lineage to its data and code versions.
3. Deploy behind a feature flag, in shadow mode, and compare its outputs with the incumbent on real traffic for a day.
4. Canary to 5% with an automatic rollback on a latency or error threshold.
5. Wire monitoring for operations, data quality, drift and business metric.
6. Write the runbook for the two most likely alerts.
7. Roll it back. Confirm that the model, preprocessing and config all revert together.

COMMON PITFALL | The five operational failures

1. **Silent degradation.** No quality monitor; users notice before you do.
2. **Skew after rollback.** Model reverted, preprocessing not.
3. **Unreproducible training.** A regression appears and cannot be bisected.
4. **Alerting on drift.** Features move constantly; you train the team to ignore alerts.
5. **No cost attribution.** Spend triples and nobody can say which feature did it.

NOTE | Time to competence

6–8 weeks for ESSENTIAL and CORE if you have one system to practise on. The reproducibility test in step 1 above is the fastest way to find out where you actually stand.

CHAPTER RECAP

- ML systems fail quietly and plausibly, which is why quality sampling, drift monitoring and shadow deployment are not overhead.
- Track every run with code, data, config and environment; the test is whether you can regenerate a three-month-old result.
- Version and roll back the model, preprocessing and config together.
- Shadow first, canary second, automatic rollback always; feature-flag every LLM feature.
- Monitor operations, data, drift, quality and business; alert on quality and business, dashboard the rest.
- LLM-specific: prompt versioning with an eval gate, tracing with cost per span, pinned model versions, spend limits.

Hardware and Systems

Enough of the machine to explain a number you did not expect

ADVANCED

PREREQS: Chapters 5, 9

You do not need to write CUDA to be an excellent ML engineer. You do need enough of the hardware model to compute a memory budget before starting a job, to explain why a kernel is slow, and to tell whether an optimisation can possibly help. This chapter is the minimum that pays for itself, plus the specialist extension.

WHAT YOU WILL LEARN

- Compute memory and throughput budgets before running anything
- Place a workload on the roofline and decide what kind of optimisation helps
- Read a profile and identify the bottleneck
- Know what to learn if you want to go deeper into kernels

The capabilities

CHECKLIST | CORE — the mental model

- The memory hierarchy on an accelerator: registers, shared memory/SRAM, L2, HBM, host memory over PCIe. Roughly an order of magnitude of latency and capacity between each
- Threads, warps and blocks; why divergent branches inside a warp serialise
- Coalesced memory access: consecutive threads must touch consecutive addresses or traffic multiplies
- Tiling: load a block into fast memory once and reuse it many times. This is what makes GEMM, convolution and attention fast
- **Arithmetic intensity** (FLOPs per byte moved) and the roofline: below the ridge point, only reducing bytes helps; above it, only reducing arithmetic helps. Compute it before optimising anything
- Why FlashAttention does *more* arithmetic and is several times faster

CHECKLIST | CORE — the arithmetic you should be able to do on paper

- Model memory: $\approx 16P$ bytes for mixed-precision training with Adam
- KV cache: $4 \cdot B \cdot S \cdot L \cdot H_{kv} \cdot d_h$ bytes in fp16
- Training compute: $\approx 6PN$ FLOPs for P parameters and N tokens
- Inference: $\approx 2P$ FLOPs per token plus attention
- Communication: ring all-reduce sends $2(P-1)/P \times N$ bytes per worker
- Time estimate: max (FLOPs/peak, bytes/bandwidth), then divide by an achieved-efficiency factor of 0.3–0.6

CHECKLIST | CORE — profiling

- `torch.profiler` with a trace viewer; Nsight Systems for the whole timeline and Nsight Compute for a single kernel
- Reading a timeline: gaps mean you are input-bound or synchronising; long memory-copy bars mean the transfer path is wrong
- Finding synchronisation points: `.item()`, `.cpu()`, `print` of a tensor, and data-dependent shapes such as `nonzero`
- Distinguishing compute-bound, memory-bound, input-bound and communication-bound with a measurement

CHECKLIST | CORE — the practical levers

- Mixed precision, and preferring `bf16` where available
- `torch.compile`, and counting graph breaks
- Kernel fusion to remove intermediate round trips to HBM
- Choosing shapes that suit tensor cores (multiples of 8 or 16 in the relevant dimensions)
- Pinned memory and a separate copy stream for host-to-device transfer
- CUDA graphs for workloads with many small kernels

CHECKLIST | SPECIALIST — if you want to go deeper

- **Triton**: a Python-embedded language for writing GPU kernels at a much gentler gradient than CUDA. This is the right entry point in 2026 and is enough to write a fused kernel that beats the framework's default
- CUDA C++ for the last increment of control
- Kernel autotuning, occupancy analysis, shared-memory bank conflicts
- Interconnect topology: NVLink within a node, InfiniBand or Ethernet between; why collective performance is topology-dependent
- TPUs and the XLA compilation model, if you work in that ecosystem

KEY INSIGHT | Why this pays for an ordinary engineer

Not so you can write kernels. So that you can answer four questions without running an experiment: *Will this fit? Is this optimisation capable of helping? Which resource am I short of? Is this number plausible?* Being able to answer those on paper saves days of speculative work per quarter, and it is a small amount of material.

How to tell you have it

PRACTICE | The tasks

1. For a model you use, compute on paper: training memory, KV cache at your context length, inference FLOPs per token, and the expected tokens per second on your hardware. Then measure all four and explain every gap larger than 2x.
2. Take a training loop at low GPU utilisation and find the bottleneck with a profile rather than a guess.
3. Compute the arithmetic intensity of five kernels you run and place them on the roofline for your device.
4. Write one fused kernel in Triton — a fused bias-plus-activation is enough — and measure it against the unfused PyTorch version.

COMMON PITFALL | Optimising the wrong thing

The dominant failure here is spending a week on a kernel when the job was input-bound, or quantising a model when the queue was the bottleneck. The discipline is unvarying: measure, place it on the roofline, then decide what class of optimisation can possibly help. An optimisation that reduces arithmetic on a memory-bound kernel is guaranteed to do nothing, and you can know that before you start.

NOTE | Time to competence

3–4 weeks for CORE, which is the part with the high return. **3–6 months** for the SPECIALIST list, and worth it only if you are aiming at research engineering or ML platform work.

CHAPTER RECAP

- Learn the hardware model to answer four questions on paper: will it fit, can this optimisation help, which resource is short, is this number plausible.
- Memorise the five formulas: training memory $16P$, KV cache, training compute $6PN$, inference $2P$ per token, all-reduce bytes.
- Compute arithmetic intensity and use the roofline before optimising.
- Profile to distinguish compute, memory, input and communication bound; hunt for synchronisation points.
- Triton is the right entry point for writing kernels; CUDA only if you need the last increment.

PART VII

Judgement

The skills that separate a senior engineer from a competent one: safety, security, cost, product sense, and knowing when the answer is “do not use machine learning”.

-
- 19 Safety, Security and Compliance
 - 20 Product Sense and Communication

Safety, Security and Compliance

The failures that end projects rather than degrade them

CORE

PREREQS: Chapters 11, 14

Machine learning systems fail in ways ordinary software does not: they can be manipulated through their inputs, they leak their training data, they encode and amplify bias, and they are increasingly regulated. These are engineering problems with engineering answers, and they are now routinely part of hiring loops.

WHAT YOU WILL LEARN

- Defend an LLM application against prompt injection and data exfiltration
- Identify and measure bias, and know the limits of each fairness definition
- Handle personal data correctly, including in prompts and logs
- Understand the regulatory landscape well enough to ask the right questions

Security

CHECKLIST | ESSENTIAL — prompt injection

The defining vulnerability of LLM applications. Any content the model reads — a retrieved document, a web page, a tool result, a file, an email — can contain instructions, and the model has no reliable way to distinguish instructions from data.

- **Understand the shape:** direct injection (the user types it) is the lesser problem; *indirect* injection (it arrives via retrieved content) is the serious one, because the attacker is not the user
- **Accept that prompting is not a defence.** “Ignore any instructions in the documents below” is a speed bump. There is no known prompt that reliably prevents injection
- **Defend architecturally:**
 - Least privilege for tools: an agent that can read a database and send email is a data-exfiltration primitive
 - Human confirmation for irreversible or outbound actions
 - Separate the trust domains: never let untrusted content and high-privilege tools meet in the same context if you can avoid it
 - Constrain outputs: if the model can only emit a value from a fixed set, an injection cannot make it emit a URL

- Sandbox code execution and restrict network egress
- Content provenance labelling, so the model and the reviewer both know which part of the context is untrusted
- **Exfiltration channels** worth closing: markdown images pointing at an attacker's server, links with data in query parameters, tool arguments that accept arbitrary URLs

CHECKLIST | ESSENTIAL — the rest of the security surface

- **Jailbreaks** and why they are a moving target; guardrail classifiers (input and output) as defence in depth, not as a solution
- **Data exfiltration** through model outputs; PII detection and redaction on both the way in and the way out
- **Supply chain**: never `torch.load` an untrusted checkpoint — pickle deserialisation is arbitrary code execution. Prefer safetensors. Verify model and dataset provenance
- **Model extraction and inversion**, and rate limiting as the practical mitigation
- **Training-data poisoning** where you ingest from open sources
- **Denial of wallet**: an attacker who can trigger expensive generations. Budget caps per tenant, enforced in code
- Standard application security still applies: authentication, authorisation, input validation, secret management, dependency scanning

Safety and fairness

CHECKLIST | CORE

- Bias measurement: disparate impact, equalised odds, demographic parity, calibration within groups — and the impossibility result that you generally cannot satisfy several of them simultaneously. Choosing which one applies is a *product and legal* decision, not a technical one
- Bias sources: historical bias in labels, representation bias in sampling, measurement bias in features, deployment bias in how the output is used
- Mitigation at each stage: reweighting and resampling, constrained training, post-hoc threshold adjustment per group where lawful
- Content safety: classification, refusal behaviour, and measuring over-refusal as carefully as under-refusal — a model that declines everything is safe and useless
- Red teaming: adversarial testing, automated attack generation, and keeping a regression suite of successful attacks
- Human oversight: where a human must be in the loop, and designing that so it is meaningful rather than rubber-stamping

Privacy and compliance

CHECKLIST | CORE

- PII detection, minimisation, redaction and pseudonymisation — including in prompts, in logs and in traces, which is where it most often leaks
- Data residency, retention and deletion; the right to erasure and what it means for a model trained on the deleted data
- Consent and lawful basis for training on user data
- Differential privacy conceptually: the privacy budget, and the accuracy cost
- Federated learning at awareness level
- Vendor terms: what a model provider may do with your prompts and outputs, and whether your data crosses a boundary you promised it would not

CHECKLIST | CORE — regulation

- **EU AI Act:** the risk-tier structure (prohibited, high-risk, limited, minimal), obligations on general-purpose models, and phased application dates. If you serve the EU, you need to know which tier your system is in
- **GDPR** where personal data is involved: lawful basis, automated decision-making provisions, data subject rights
- Sector rules: HIPAA for health, financial services model risk management (SR 11-7 and equivalents), and any regulator-specific model documentation
- **Documentation as a deliverable:** model cards, data sheets, intended use, limitations, evaluation results. Increasingly required rather than recommended, and a good practice regardless
- Provenance and content labelling obligations for synthetic media

WATCH OUT | The one that catches engineers

Regulatory exposure attaches to *use*, not to the model. The same classifier is unregulated in a recommendation feature and high-risk in a hiring or credit decision. Ask “what decision does this output affect, about whom?” early — before the architecture is fixed — because the answer changes your documentation, evaluation and human-oversight obligations substantially.

How to tell you have it

PRACTICE | The tasks

1. Build an agent with a retrieval tool and an outbound tool (email or HTTP). Then write an injection that lives in a retrieved document and exfiltrates data. Then fix it architecturally, not by prompting, and demonstrate the attack now fails.
2. Take a classifier and measure disparate impact and equalised odds across a protected attribute. Show that adjusting thresholds to equalise one metric moves the other, and write down which you would choose and why.
3. Run PII detection over your prompt and trace logs. Report what you find; there is almost always something.
4. Write a model card for something you built: intended use, out-of-scope use, training data, evaluation results, limitations, and the failure modes you know about.

COMMON PITFALL | Four mistakes with serious consequences

1. **Treating prompt injection as a prompting problem.** It is an architecture problem. Every prompting mitigation has been bypassed.
2. **Logging prompts containing PII.** The most common privacy incident in LLM applications, and entirely avoidable.
3. **`torch.load` on a downloaded checkpoint.** Remote code execution. Use safetensors.
4. **Optimising only for under-refusal.** A system that refuses ten per cent of legitimate requests has a serious product defect that no safety metric will show you.

NOTE | Time to competence

3–4 weeks for working competence across ESSENTIAL and CORE, plus ongoing attention because the attack surface and the regulation both move.

CHAPTER RECAP

- Indirect prompt injection is the defining LLM vulnerability and cannot be fixed by prompting; defend with least privilege, confirmation gates, trust separation, constrained output and sandboxing.
- Close exfiltration channels: markdown images, links with parameters, arbitrary URLs in tool arguments.
- Never load untrusted pickled checkpoints; prefer safetensors.
- Fairness definitions conflict mathematically; choosing one is a product and legal decision that you should make explicitly.
- Measure over-refusal as carefully as under-refusal.
- PII leaks through logs and traces more often than through outputs.
- Regulatory exposure attaches to the use, not the model; ask what decision it affects and about whom, early.

Product Sense and Communication

The skills that decide whether the work mattered

CORE

PREREQS: none

The most common way for a technically excellent machine learning project to fail is for it to solve the wrong problem, or to solve the right one in a way nobody adopts. These are learnable skills with concrete practices, and they are the clearest difference between a mid-level engineer and a senior one.

WHAT YOU WILL LEARN

- Frame a business problem as a machine learning problem, or decline to
- Choose a metric that a stakeholder and an engineer both understand
- Write a design document that gets a decision made
- Communicate uncertainty without either overclaiming or being useless

Problem framing

CHECKLIST | ESSENTIAL — the questions to ask first

1. **What decision does this output change?** If the answer is “none”, stop. A prediction nobody acts on has no value.
2. **What happens today?** There is always an incumbent: a rule, a human, a spreadsheet. That is your baseline, and it is often better than expected.
3. **What is the cost of each error type?** A false positive and a false negative are almost never equally expensive, and the ratio determines your metric and your threshold.
4. **What accuracy would be good enough to act on?** If nobody can answer this, the project has no success criterion.
5. **Is there enough of the right data, with labels, from the right distribution?** Ask before promising anything.
6. **What is the latency and cost budget?** These constrain the solution space more than accuracy usually does.
7. **How will it be wrong, and who is harmed?** Chapter 19.

CHECKLIST | ESSENTIAL — when not to use machine learning

- A rule would work. Rules are explainable, testable, instant and free
- You cannot define success
- You do not have labels and cannot obtain them
- The cost of an error is catastrophic and unrecoverable
- The problem changes faster than you could retrain
- The data does not exist yet — in which case the project is instrumentation, not modelling

Saying this clearly and early is a senior behaviour and is consistently valued more than delivering a model nobody needed.

Metrics that connect

CHECKLIST | ESSENTIAL

- Build the chain explicitly: model metric → product metric → business metric. If you cannot draw that chain, you cannot argue for the work
- Choose a single primary metric and a small set of guardrails (latency, cost, fairness, safety) that must not degrade
- Distinguish offline and online metrics, and expect them to disagree — when they do, the online one is right and the gap is information
- Sanity-check magnitude: a claimed 40% improvement in a mature system is almost always a bug or a leak
- Present a confidence interval, always

Writing and speaking

CHECKLIST | ESSENTIAL — the design document

A one-to-three page document, written *before* the work, containing: problem, current state, proposed approach, alternatives considered and why rejected, success criteria, risks, and how you will know it failed. Its purpose is to get a decision made and to be re-readable in a year.

The section people skip and should not is *alternatives considered*. It is what turns a proposal into a defensible decision.

CHECKLIST | ESSENTIAL — communicating results

- Lead with the conclusion and the decision you want. Then the evidence
- State uncertainty in terms of consequences: “we are confident it is between 2% and 6% better, which at current volume is between X and Y” beats a p-value
- One chart per claim, with axes labelled and a confidence interval drawn
- Translate: “AUC 0.84” means nothing to most audiences; “catches 70% of fraud while flagging 2% of good transactions” means something
- Report the failures and limitations yourself, before somebody finds them. It is the fastest way to become trusted

CHECKLIST | CORE — working with others

- With product: negotiate the accuracy/latency/cost triangle explicitly, and show them the frontier rather than a single point
- With engineering: agree the interface and the failure behaviour early — what does the service return when the model is unavailable
- With legal and compliance: bring them in before the architecture is fixed, not at launch
- With annotators and domain experts: they know things the data does not encode; their disagreements are information about your guidelines
- Code review and design review given kindly and specifically
- Mentoring: the fastest way to consolidate your own knowledge

CHECKLIST | CORE — staying current without drowning

- Read a small number of primary sources rather than a large number of summaries. Model and system cards, and the papers behind the tools you actually use
- Reproduce one technique a month, in code. Reading a paper produces recognition; implementing it produces capability
- Follow the release notes of the three or four libraries you depend on — this is where genuinely useful changes appear
- Keep a written log of what you tried and what happened. It is the only defence against relearning the same lesson
- Deliberately ignore most of it. The volume of AI content is now far beyond what anyone can track, and the fraction that will matter in a year is small

KEY INSIGHT | The single most valuable professional habit

Write down what you expect *before* you run the experiment. A one-line prediction: “I expect this to improve recall by about 3% and to cost 20% more latency.” Then compare. Over a year this does two things: it calibrates your intuition faster than anything else, and it makes you notice when a result is surprising — which is where the learning and most of the bugs both live.

How to tell you have it

PRACTICE | The tasks

1. Take a project you worked on and write, retrospectively, the one-page design document you should have written. Include the alternatives.
2. Take your best result and explain it in three sentences to somebody with no technical background. Have them tell you what decision they would make on the basis of it. If they cannot, rewrite.
3. Find a project in your organisation that should not use machine learning, and write the two-paragraph argument for the rule-based alternative.
4. For a model you own, draw the full chain from model metric to business metric with the arithmetic on each arrow.

COMMON PITFALL | The four ways good work goes to waste

1. **Solving the wrong problem.** No decision changes as a result.
2. **Optimising an unconnected metric.** AUC improves, revenue does not, and nobody can explain why.
3. **Delivering without a handover.** No runbook, no documentation, no owner; it degrades and gets switched off.
4. **Overclaiming.** One overstated result costs more credibility than several good ones earn.

NOTE | Time to competence

Continuous, and the fastest route is writing. One design document and one written result summary per project, reviewed by somebody senior, improves this faster than any reading.

CHAPTER RECAP

- Ask what decision the output changes, what happens today, and what error costs, before anything technical.
- Be willing to say that machine learning is the wrong tool; it is a senior behaviour and it is valued.
- Draw the chain from model metric to business metric, pick one primary metric and explicit guardrails.
- Write a short design document before the work, including alternatives considered.
- Lead with the conclusion, express uncertainty in consequences, and report your own limitations first.
- Write down your expected result before running the experiment; nothing calibrates judgement faster.

PART VIII

Getting There

What changed recently, what is now obsolete, four concrete learning tracks with week-by-week plans, portfolio projects that actually signal competence, and a self-assessment rubric.

-
- 21 What Changed, and What Is Now Obsolete
 - 22 Four Learning Tracks
 - 23 A Portfolio That Signals Competence
 - 24 Self-Assessment

What Changed, and What Is Now Obsolete

An honest account of what to learn now and what to stop learning

REFERENCE

PREREQS: none

Roadmaps written three years ago are actively misleading in places, and the parts that went stale are not the parts people expect. This chapter states plainly what became important recently, what became less important, and — the part usually missing — what did not change at all.

WHAT YOU WILL LEARN

- Know which skills became scarce and valuable recently
- Stop spending time on material that has quietly stopped mattering
- Recognise the durable core that survives every cycle

The shape of the change



The 2020 column has not disappeared — it is still half of most ML jobs. The 2026 column is what is newly scarce.

Figure 21.1 What the job asked for, by year. The left column has not disappeared — it is still most of the work in most ML roles. The right column is what is newly scarce.

What became important

CHECKLIST | The genuinely new, in rough order of how scarce the skill is

Evaluation as a first-class discipline. The single largest gap in the market. Teams can build LLM features and cannot tell whether a change helped. Chapter 15.

Inference cost and latency engineering. When serving cost is a material line item, the ability to cut it by an order of magnitude through batching, caching, quantisation and routing is directly valuable. Chapter 16.

Agents and tool use. Reliability engineering for non-deterministic loops: budgets, verification, failure taxonomies, sandboxing. Chapter 14. MCP has emerged as a cross-vendor standard for exposing tools.

Post-training. LoRA and QLoRA made fine-tuning accessible; DPO and its relatives made preference optimisation accessible without an RL stack; GRPO and verifiable rewards made reasoning-style training tractable. Chapter 13.

Retrieval done properly. Not “put documents in a vector database” but hybrid search, reranking, contextual chunking, and per-stage measurement. Chapter 12.

Data curation at scale. Dedup, filtering, mixture design, synthetic generation, contamination control. The largest reliable quality gains of recent years came from here. Chapter 7.

Prompt injection and LLM security. A genuinely new vulnerability class with no prompting-level fix. Chapter 19.

Context engineering. Managing what goes into a finite window — ordering, budgeting, compaction, externalising memory — as a distinct skill from prompt wording.

What became less important

CHECKLIST | Things worth substantially less time than they were

Hand-designing architectures. The transformer and its standard variants cover most of what practitioners need. Designing novel architectures is a research activity, not an engineering one.

Manual feature engineering for unstructured data. Superseded by learned representations. Still essential for *tabular* data, where it remains a core skill.

Classical NLP pipelines. Separate tokenisers, POS taggers, parsers, NER models, and the associated toolchain. A general model with a good prompt or a small fine-tune covers most of it, and does so better.

Grid search. Random search dominates it, and Bayesian search (Optuna and similar) dominates random. Grid search persists mostly in tutorials.

TensorFlow 1.x and Keras-first workflows. TensorFlow still runs in production and JAX is strong in research, but PyTorch is the default and the one to learn first. Multi-framework fluency is no longer a baseline requirement.

Hand-writing training loops for everything. Write one from scratch to understand it; then use `accelerate`, `trl`, Lightning or similar.

Building your own vector index. Understand how they work — it determines your memory budget and recall — but use FAISS or a vector database.

Word2vec, GloVe and static embeddings. Historically important; contextual embeddings are better on essentially every task.

WATCH OUT | Less important is not “safe to be ignorant of”

Everything in that list still appears in interviews and in legacy systems. The claim is that it deserves less of your *learning* time, not that you should be unable to discuss it. Knowing why static embeddings were superseded is a two-minute answer that demonstrates understanding; being unable to recognise word2vec suggests a gap.

What did not change

CHECKLIST | The durable core

- **Software engineering.** Version control, testing, packaging, CI, reproducibility. More important now, not less, because AI systems are harder to make reproducible.
- **Data quality.** Leakage, train–serve skew, distribution shift. Identical failures, now with more expensive models on top.
- **Evaluation discipline.** Golden sets, held-out data, confidence intervals, calibration. The techniques transferred wholesale from classical ML to foundation models — which is why people with a classical background are noticeably better at LLM evaluation.
- **Gradient-boosted trees on tabular data.** Still the right answer. Still winning. Still most of the ML in most companies.
- **The memory hierarchy.** Sequential beats random; bytes moved beats operations performed. FlashAttention is this principle, applied.
- **Problem framing.** What decision does this change; what happens today; what does an error cost.
- **Mathematics.** Linear algebra, probability, optimisation. Unchanged and unchanging.
- **Distributed systems.** Ring all-reduce’s bandwidth optimality is a fact about topology, not about a framework.

KEY INSIGHT | How to read a roadmap, including this one

Separate three categories every time: **principles** (memory hierarchy, evaluation discipline, problem framing) which last decades; **techniques** (LoRA, DPO, paged attention, hybrid retrieval) which last years; and **tools** (a specific library or vendor) which last months. Spend your learning time in that order of priority, and treat the tool layer as something you pick up in a day when you need it.

A roadmap that is mostly tool names is telling you about its author’s recent month, not about the field.

Where the demand actually is

Area	Supply of engineers	Demand	Comment
Building LLM demos	very high	saturated	the tutorial ceiling

Area	Supply of engineers	Demand	Comment
LLM evaluation	very low	high	the clearest opportunity
Inference cost engineering	low	high and growing	directly measurable value
ML platform / infrastructure	low	high	persistently underserved
Data curation at scale	low	high	quiet, valuable, unglamorous
Classical ML in production	moderate	steady and large	undervalued by newcomers
Frontier research	very high	very low	extremely competitive

CHAPTER RECAP

- Newly scarce: evaluation, inference cost engineering, agent reliability, post-training, retrieval done properly, data curation, LLM security, context engineering.
- Worth less time now: novel architectures, manual features for unstructured data, classical NLP pipelines, grid search, TensorFlow-first workflows, building your own index, static embeddings.
- Unchanged: software engineering, data quality, evaluation discipline, GBDT on tabular data, the memory hierarchy, problem framing, mathematics, distributed systems.
- Separate principles from techniques from tools, and spend your time in that order.
- Demand is highest where the work is rigorous and unglamorous.

Four Learning Tracks

Week-by-week plans from four realistic starting points

PRACTICE

PREREQS: Chapter 1

Generic roadmaps fail because they ignore where you are starting from. These four tracks assume different backgrounds, aim at different roles, and give week-by-week plans with a deliverable each week. Pick the one whose starting point matches yours, and do not mix them.

WHAT YOU WILL LEARN

- Choose a track matched to your background and target role
- Follow a week-by-week plan with a concrete weekly deliverable
- Know what to skip, which is as important as what to study

The tracks

A. Software engineer → AI engineer	3–4 months	Ch 3, 11–16, 19–20
B. Data scientist → ML engineer	4–6 months	Ch 3, 5, 6, 9, 16–18
C. Student / career changer	9–12 months	all of it, in order
D. ML engineer → research engineer	6–9 months	Ch 4, 9, 10, 13, 18

Elapsed time at roughly 12 hours a week. Every track assumes you build and ship something real; none of them works by reading alone.

Figure 22.1 Four paths. Durations assume roughly 12 hours a week and that you build something real throughout; none of them works by reading alone.

NOTE | The rule that applies to every track

Each week has a **deliverable**: something that runs, that you could show somebody. A week without a deliverable did not happen. If you are behind, cut the reading, never the building.

Track A — Software engineer to AI engineer

Assumes: you write production software, use git and Docker daily, and have written tests. **Target:** building LLM-powered products. **Duration:** 12–16 weeks.

Week	Focus	Deliverable
1	Ch 4 maths, the ESSENTIAL list only	Backprop by hand for a 2-layer MLP, matched to PyTorch
2	Ch 11 LLM fundamentals	Tokenisation and cost analysis of your own documents across three models
3	Ch 11 tool use, structured output	Extraction service hitting 95% schema validity
4–5	Ch 15 evaluation	A golden set, three graders, a validated judge, CI integration
6–7	Ch 12 retrieval	RAG pipeline with hybrid search and a reranker, plus per-stage metrics
8	Ch 5 algorithms, interview patterns	25 problems, patterns named
9–10	Ch 14 agents	An agent with budgets, three tools, a trajectory eval and a failure taxonomy
11	Ch 19 security	Injection attack on your own agent, then an architectural fix
12	Ch 16 serving basics	Cost and latency model; routing rule with measured savings
13–16	Ch 8 classical ML, Ch 17 MLOps, portfolio	One tabular project end to end; deploy the agent with monitoring

Track B — Data scientist to ML engineer

Assumes: statistics, sklearn, pandas, SQL. **Target:** owning models in production. **Duration:** 16–24 weeks.

Week	Focus	Deliverable
1–3	Ch 3 engineering	A notebook converted into a tested, typed, containerised package with CI
4–5	Ch 5 algorithms	30 problems; explain the row-versus-column traversal gap
6–7	Ch 6 data engineering	Point-in-time-correct feature pipeline with validation and a skew test
8	Ch 8 classical ML, the production half	Calibration, cost-matrix thresholds, SHAP explanation to a non-specialist
9–12	Ch 9 deep learning	Transformer from scratch; overfit a batch; induce and diagnose six bugs
13–14	Ch 17 MLOps	Registry, shadow deploy, canary with auto-rollback, monitoring, runbook

Week	Focus	Deliverable
15–16	Ch 11–12 LLM and retrieval	RAG pipeline with per-stage evaluation
17–18	Ch 15 evaluation	Harness for both a classical and an LLM system
19–24	Ch 10, 16, 18 plus a capstone	Distributed fine-tune; serving with measured cost; one full system

Table 22.1 Track A. You are skipping deep learning internals (Ch 9–10) deliberately; add them later if you move toward ML engineering.

Track C — Student or career changer

Assumes: some programming, no production experience. **Target:** a first role. **Duration:** 36–48 weeks. Be honest with yourself about this number.

Weeks	Focus	Deliverable
1–6	Ch 3 Python and engineering	Three packaged, tested projects; git fluency
7–12	Ch 4 maths	Backprop by hand; bootstrap confidence intervals; a probability notebook
13–18	Ch 5 algorithms	75 problems across all patterns; the DSA companion volume
19–22	Ch 6 data	SQL fluency; a pipeline with validation and backfill
23–26	Ch 8 classical ML	Two tabular projects, deployed, with honest evaluation
27–32	Ch 9 deep learning	Transformer from scratch; a fine-tune; the debugging playbook
33–38	Ch 11–15 foundation models	RAG system, agent, and an evaluation harness for both
39–44	Ch 16–17 production	Serve something; monitor it; write the runbook
45–48	Portfolio and applications	Three polished projects, written up; Ch 23

Table 22.2 Track B. The engineering weeks at the start are the ones people want to skip and the ones that matter most.

Track D — ML engineer to research engineer

Assumes: you train and deploy models. **Target:** implementing and scaling research. **Duration:** 24–36 weeks.

Weeks	Focus	Deliverable
1–4	Ch 4 maths, deeper	Derive attention gradients; read three papers' derivations
5–10	Ch 9 deep learning internals	Reimplement three papers from scratch and match reported numbers

Weeks	Focus	Deliverable
11–16	Ch 10 training at scale	Multi-node FSDP run; profile; fix a communication bottleneck
17–22	Ch 18 hardware	Triton kernels; roofline analysis; beat a framework default
23–28	Ch 13 post-training	SFT, DPO and a GRPO-style run with verifiable rewards
29–36	Original work	Reproduce a recent result, then extend it; write it up

Table 22.3 Track C. The temptation is to start at week 33. Everybody who does so stalls, because the foundations weeks are what make the later weeks productive rather than imitative.

COMMON PITFALL | The four ways tracks fail

1. **Mixing tracks.** Doing a bit of each produces breadth with no depth and no portfolio.
2. **Reading without building.** Recognition feels like knowledge and disappears.
3. **Starting in the middle.** Track C's weeks 1–18 are what make weeks 33–48 productive rather than imitative.
4. **No deadline.** A track without dates becomes indefinite. Put the dates in a calendar and treat them as real.

KEY INSIGHT | If you have less time than the track assumes

Halve the scope, not the rigour. Better to do half the weeks properly, with deliverables, than all of them superficially. The deliverables are what you show an employer and what you actually retain; the reading is the scaffolding.

CHAPTER RECAP

- Four tracks from four starting points; pick one and do not mix.
- Every week has a deliverable; a week without one did not happen.
- Track A (SWE to AI engineer) is the shortest at 12–16 weeks; Track C (career change) is honestly 36–48.
- If you are behind, cut reading, never building.
- Put real dates in a calendar; a track without deadlines becomes indefinite.

A Portfolio That Signals Competence

What to build, and how to write it up so it is believed

PRACTICE

PREREQS: Chapter 22

Most portfolios are indistinguishable from tutorials, which is why they do not work. This chapter describes what a project has to contain to be evidence of engineering ability, gives six project specifications at increasing difficulty, and shows how to write them up so a reviewer with four minutes reaches the right conclusion.

WHAT YOU WILL LEARN

- Understand what a reviewer is actually checking for in four minutes
- Build three projects that are evidence rather than imitation
- Write a README that makes the evidence legible

What a reviewer is checking

A hiring manager spends about four minutes on a portfolio project. They are not reading your model code. They are looking for evidence of six things:

CHECKLIST | The six signals

1. **Does it run?** Clear setup, pinned dependencies, and it works on a machine that is not yours.
2. **Is it tested?** Any tests at all put you ahead of most candidates. Tests of the data logic put you well ahead.
3. **Is the evaluation honest?** A held-out set, a stated baseline, an interval or a significance statement, and at least one place where you say what did not work.
4. **Did you make decisions?** A trade-off named and resolved, with the reasoning visible.
5. **Can you write?** The README is the sample of your communication that gets read.
6. **Is it yours?** Not a tutorial with the dataset swapped.

COMMON PITFALL | Why the standard portfolio fails

Titanic, MNIST, IMDB sentiment, “chat with your PDF”. These are all *tutorial completions*. They demonstrate that you can follow instructions, which was never in question. The signal is zero because there is no decision in them — every choice was made by the tutorial author.

Adding a Streamlit front end does not fix this. The problem is not presentation, it is that no judgement was exercised.

Six projects

Each specification below has a decision embedded in it that the project cannot be completed without making.

1. An evaluation harness for an LLM feature. Pick a task. Build a golden set of 150–300 labelled cases, including adversarial and edge cases. Implement three grader types (exact/programmatic, similarity, LLM-as-judge). Validate the judge against human labels and report the agreement rate. Wire it into CI so a regression fails a build. Write up which grader disagreed with you and what you did about it. *The decision:* what counts as correct, and how you know your grader knows.

2. A RAG system with per-stage measurement. A corpus of at least 5,000 documents. Measure retrieval recall@k separately from answer quality. Compare at least three configurations (dense only; hybrid; hybrid plus reranker) with numbers. Report latency and cost per query alongside quality. *The decision:* where the failures actually are — and the answer must come from your own measurements, not from a blog post.

3. A tabular model in production form. A real dataset with temporal structure. Point-in-time-correct features, a time-based split, calibration, a threshold chosen from an explicit cost matrix, SHAP explanations, a served endpoint, and monitoring with drift detection. *The decision:* the operating threshold, justified in the currency of the problem rather than by maximising F1.

4. An agent with a failure taxonomy. A multi-step task with three or more tools. Step budget, token budget, timeout. Trajectory-level evaluation over at least 50 tasks. Then the part that matters: categorise every failure, count the categories, fix the largest one, and show the before-and-after numbers. *The decision:* which failure mode to spend your effort on.

5. An inference cost reduction. Take a working LLM feature. Measure baseline cost and p95 latency. Apply at least three techniques from Chapter 16 — caching, batching, quantisation, model routing, speculative decoding. Report the cost curve and the quality change at each step. *The decision:* how much quality you are willing to trade for how much cost, stated up front.

6. A fine-tune with a real baseline. Pick a task where a strong prompted baseline exists. Establish that baseline properly. Then fine-tune — LoRA or QLoRA — and show whether you beat it, on a held-out set, with the cost of both approaches stated. *The decision:* whether fine-tuning was worth it. “No” is a perfectly good result and a rarer, more credible write-up than yet another claimed win.

KEY INSIGHT | Three is the right number

Three projects at this depth beat ten shallow ones, and beat one enormous one. Three demonstrates range; depth demonstrates ability; more than three demonstrates that you did not know when to stop.

Pick them to span your target role: for an AI engineer, projects 1, 2 and 4; for an ML engineer, 3, 6 and 1; for an inference or platform role, 5, 3 and 2.

The write-up

A reviewer's four minutes are spent on the README. Structure it so the first screen answers the question they are actually asking.

CHECKLIST | README structure

Problem (2 sentences). What decision does this system support, and what happened before it existed.

Result (1 short table). The headline numbers against a named baseline, with the metric defined.

Approach (1 paragraph plus one diagram). What you built. One image.

Trade-offs (3–5 bullets). What you chose and what you gave up. This is the section that separates a portfolio from a tutorial.

What did not work (3 bullets). The most credible section in any write-up. Reviewers read it first once they know it is there.

Evaluation (short section). Dataset, split, metrics, intervals. State the sample size. State whether the difference is significant.

Run it (a code block). Two commands, maximum.

Limitations (3 bullets). Where it breaks and what you would do next.

1	## Result				
2					
3	Configuration	Recall@10	Answer acc.	p95 latency	\$/1k
4	queries				
5	----- ----- ----- ----- -----				
6	-----				
7	Dense only (baseline)	0.61	0.58	340ms	
8	1.20				
9	+ BM25 hybrid	0.78	0.67	390ms	
10	1.25				
11	+ cross-encoder rerank	0.89	0.74	720ms	
12	2.10				
13	n = 240 held-out questions; answer accuracy graded by a judge validated at				
14	0.91 agreement with human labels on a 60-item sample.				

NOTE | What this table does

It names a baseline, separates retrieval from generation, shows the cost of each quality gain, and states how the grader was validated. A reviewer who reads only this block already knows you can do the job. Everything below it is confirmation.

WATCH OUT | Things that actively hurt

- Claimed accuracy with no baseline and no split description.
- A notebook as the only artefact.
- `requirements.txt` with no versions.
- A demo link that is down. Better no link than a dead one.
- “State of the art” anywhere in the text.
- Screenshots of a chat interface used as evidence of quality.

Where the projects live

Public repository, one per project, each with the README above. A short write-up per project on a personal page or a blog, aimed at an engineer rather than a recruiter. Do not build a portfolio *website* — reviewers read repositories. Time spent on the site is time not spent on the third project.

CHAPTER RECAP

- Reviewers check six things in four minutes: it runs, it is tested, the evaluation is honest, decisions were made, you can write, it is yours.
- Tutorial completions signal nothing because no judgement was exercised.
- Build three projects spanning your target role, each containing a real decision.
- The README carries the signal: problem, result table, approach, trade-offs, what did not work, evaluation, run it, limitations.
- “What did not work” is the most credible section you can write.

Self-Assessment

A rubric for knowing where you actually are

REFERENCE

PREREQS: none

Self-assessment in this field is unreliable in a specific direction: people who have read a great deal overestimate themselves, and people who have shipped systems underestimate themselves. This rubric replaces “do I know X” with “can I do X, unaided, right now”, which is the only question with a checkable answer.

WHAT YOU WILL LEARN

- Place yourself honestly on a five-level scale per area
- Identify the one or two gaps worth closing next
- Re-assess on a schedule rather than by feel

The levels

1	Aware	can define it, has read about it
2	Guided	can do it following a tutorial
3	Independent	can do it on a new problem, unaided
4	Fluent	knows the failure modes and the trade-offs
5	Authoritative	others ask you; you have changed the default

Hiring bars, roughly: junior wants 3s in the foundations; mid wants 3–4 across a whole column of the role map; senior wants a 5 somewhere and no 1s anywhere that touches production.

Figure 24.1 Five levels. The jump that matters for employment is 2 to 3: from following a path to handling a situation nobody prepared you for.

Level	Name	Test
1	Aware	You can define the term and say why it matters
2	Guided	You can do it with documentation and an example open

Level	Name	Test
3	Independent	You can do it unaided, and debug it when it fails
4	Fluent	You can choose between approaches and justify the choice with numbers
5	Authoritative	You can teach it, extend it, and know where the standard advice is wrong

KEY INSIGHT | Use verbs, not nouns

“I know transformers” is unfalsifiable. “I can implement multi-head attention from scratch, unaided, and explain why the scaling factor is $1/\sqrt{d_k}$ ” is checkable in ten minutes. Every row below is phrased as something you either can or cannot currently do. If you are unsure, the answer is no — a skill you are unsure about is not one you can rely on under pressure.

The rubric

Score each row 1–5. Be strict; the rubric is only useful if it is honest.

#	Skill	Chapter
1	Write a tested, typed, packaged Python library others install	3
2	Use git properly: bisect, rebase, resolve a real conflict	3
3	Derive backprop for a 2-layer network by hand	4
4	Explain why a matrix–vector product is memory-bound	4, 18
5	Compute a bootstrap confidence interval and say what it means	4, 15
6	Solve a medium interview problem in 25 minutes, naming the pattern	5
7	Write correct SQL with window functions over a large table	6
8	Build a point-in-time-correct feature pipeline	6
9	Detect and fix train–serve skew	6, 17
10	Deduplicate and quality-filter a large text corpus	7
11	Beat a GBDT baseline on tabular data, or explain why you cannot	8
12	Calibrate a classifier and pick a threshold from a cost matrix	8
13	Implement a transformer block from scratch	9
14	Debug a model that trains but does not learn	9
15	Run a multi-GPU training job with FSDP and diagnose a stall	10
16	Estimate memory for a given model, batch size and optimiser	10
17	Explain KV cache size and how GQA changes it	11, 16
18	Build a RAG pipeline and measure each stage separately	12
19	Choose between prompting, RAG, fine-tuning and pre-training, with reasons	13

#	Skill	Chapter
20	Run a LoRA fine-tune and beat a prompted baseline, or show you did not	13
21	Build an agent with budgets and a failure taxonomy	14
22	Build a golden set and validate an LLM judge against humans	15
23	Cut inference cost by 5x with quality measured at each step	16
24	Deploy a model with canary, monitoring and automatic rollback	17
25	Read a roofline chart and say whether a kernel is worth optimising	18
26	Demonstrate prompt injection on your own system and fix it architecturally	19
27	Explain a model's decision to a non-technical stakeholder	20
28	Tell a stakeholder their project should not use ML	20

Table 24.1 The level scale. Note that level 3 requires debugging, not just building — a system that works only when nothing goes wrong is level 2 evidence.

Reading your scores

CHECKLIST | What the totals mean — as a rough guide only

- **Under 60:** foundations phase. Work Track C's early weeks. Do not skip to foundation models; it will not stick.
- **60–90:** junior-capable. You can contribute under supervision. Push two or three areas to level 3 and build the first portfolio project.
- **90–120:** mid-level. Depth in some areas, gaps in others. Fill the gaps that block your target role, ignore the rest for now.
- **120–140:** senior-capable in your specialisation. Your constraint is probably rows 27–28, not the technical rows.
- **Above 140:** verify with evidence. Every level 4 and 5 should correspond to something you have shipped or taught. If it does not, it is a 3.

WATCH OUT | The two reliable biases

Reading inflation. Having read about FSDP thoroughly is level 1–2, not 4. The test for 3 is “unaided, and debug it when it fails”, and nothing you read gets you there.

Shipping deflation. Having run something in production for a year while feeling you never learned it properly is usually level 4. If you have diagnosed its failures at 3am, you are past 3.

These two biases mean the loudest self-assessments are often the least accurate ones.

Turning the rubric into a plan

1. Mark the five lowest rows that your **target role** actually needs. Use the role table in Chapter 1; an AI engineer does not need row 15 at level 4.

2. For each, write the concrete deliverable that would move it up one level. Not “study FSDP” — “run a two-node FSDP job and fix one stall”.
3. Schedule them. Two per month is a realistic rate alongside a job.
4. Re-score every three months, not more often. Skills move slowly and frequent re-scoring measures mood rather than ability.

KEY INSIGHT | The last two rows

Rows 27 and 28 — explaining a decision to a stakeholder, and telling someone not to use ML — are scored lowest by almost everyone and are the two that most reliably separate a senior engineer from a strong mid-level one. They are also the fastest to improve, because the practice is free: explain your last project to someone outside the field and watch where they stop following.

CHAPTER RECAP

- Score skills by what you can do unaided, not by what you have read.
- Level 3 requires debugging, not just building.
- Twenty-eight checkable rows, each mapped to a chapter.
- Reading inflates scores; shipping deflates them. Correct for both.
- Turn the five lowest role-relevant rows into dated deliverables, and re-score quarterly, not weekly.